

SRH Berlin University of Applied Sciences – Campus Hamburg

MSc Applied Data Science and Analytics

Department of Information

in collaboration with

Faculty of Information Technology, Brno University of Technology (VUT)

ReasonGuard

Runtime Verification of AI Reasoning Faithfulness
Against Formally Verified Bounds

*A thesis submitted in partial fulfilment of the requirements for the degree of
Master of Science in Applied Data Science and Analytics*

Author: Ravan Bilalov

Student ID: 100003562

First Supervisor: Prof. Dr. Gayan de Silva, SRH University Hamburg

Second Supervisor: Assoc. Prof. Dr. Ondřej Ryšavý, VUT Brno

Submission Date: 14 September 2026

Declaration of Authorship

I, **Ravan Bilalov**, hereby declare that this thesis, titled *ReasonGuard: Runtime Verification of AI Reasoning Faithfulness Against Formally Verified Bounds*, and the work presented in it are my own and have been generated by me as the result of my own original research. I confirm that:

- this work was carried out wholly while in candidature for a Master's degree at SRH University Hamburg, in collaboration with the Faculty of Information Technology, Brno University of Technology;
- where any part of this thesis has previously been submitted for a degree or any other qualification at this or any other institution, this has been clearly stated;
- where I have consulted the published work of others, this is always clearly attributed;
- where I have quoted from the work of others, the source is always given; with the exception of such quotations, this thesis is entirely my own work;
- I have acknowledged all main sources of help;
- I have disclosed all use of generative artificial-intelligence tools in Appendix D, in line with the SRH policy on generative AI in research, and I take full responsibility for the accuracy and integrity of the entire thesis;
- where the thesis is based on work done jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: 
Place, Date: Hamburg, 14.09.2026

Abstract

Language models are increasingly used to translate outputs of formal methods into human-interpretable justifications. However, in safety-critical settings, explanations may deviate from the verdicts they are meant to explain, containing fabricated, inconsistent, or otherwise incorrect information. In such situations, operators might be encouraged to perform an improper course of action. Existing evaluation approaches are probabilistic in nature and are applied after the explanation was generated. Such properties make them unfit for use-cases where the correctness of explanations must be verified while they are generated.

We introduce ReasonGuard, a framework for verification of natural-language explanations where the output of a formal detector serves as a source of bounds on claims made by the model. ReasonGuard classifies the generated explanations into five reasoning violation categories, V1 to V5, that represent ways in which an explanation can violate these bounds. To evaluate ReasonGuard's capabilities, we implement the framework as an open-source Python pipeline, apply it to security-relevant events in industrial control system networks, and assess explanations generated by three local and two cloud-hosted LLMs for four proxy event types from the IEC-104 dataset released by NES@FIT group of Brno University of Technology for smart-grid monitoring. Because the official NES@FIT automaton schema was not available, the evaluation is carried out in a proxy regime: the machine-checkable constraints are inactive, and both the bounds and the event-type labels are proxies produced by an internal classifier rather than by a formally verified detector. The reported results are therefore relative to this proxy and do not constitute empirical validation against the official formal schema.

Results demonstrate that violation categories are statistically (Chi-square test) associated with event types and language models. Overall, we find agreement on real outputs to be at chance level ($\kappa \approx -0.01$). We therefore did not reach our agreement goal; we regard this to be an important negative result and not a minor limitation. Nonetheless, analysing the pipeline's behaviour provides valuable insights into the capabilities of rule-based verifiers of explanation faithfulness.

The contributions of this thesis include four major achievements. A pipeline for verification of LLM-generated explanations was specified and implemented; a new taxonomy of reasoning errors was proposed, which captures common LLM pathologies such as hallucinations, paying special attention to reasoning-related ones. Third, a comparative evaluation of multiple language models was performed. Lastly, actionable recommendations were formulated to

address identified challenges. These recommendations are mapped to Articles 13 and 17 of the European Union AI Act that mandate transparency of AI systems and the establishment of quality management practices.

Keywords: *runtime verification; formal methods; large language models; faithfulness; reasoning verification; industrial control systems; IEC-104; EU AI Act; smart-grid security.*

Acknowledgements

First of all, I would like to acknowledge Prof. Dr. Gayan de Silva at SRH University Hamburg for supervising this thesis. I thank him for the valuable input about system architecture and for the briefings about formal methods upon which this thesis builds.

Furthermore, I want to thank Assoc. Prof. Dr. Ondřej Ryšavý and the NES@FIT group at the Faculty of Information Technology, Brno University of Technology, for providing the ICS datasets, the documentation of the automaton's output and the labelling by domain experts on which the evaluation of ReasonGuard is based.

I am grateful to SRH University Hamburg's Department of Information and the MSc Applied Data Science and Analytics programme for providing support during this thesis.

Last but not least, thanks to my family for their support during the whole Master's studies.

Table of Contents

Declaration of Authorship.....	1
Abstract.....	2
Acknowledgements.....	3
Table of Contents.....	4
List of Figures.....	7
List of Tables.....	8
List of Abbreviations.....	9
1 Introduction.....	10
1.1 Background and Motivation.....	10
1.2 Problem Statement.....	12
1.3 Research Questions.....	13
1.4 Significance of the Study.....	13
1.5 Scope and Delimitations.....	14
1.6 Contributions.....	15
1.7 Thesis Structure.....	15
2 Literature Review.....	16
2.1 Formal Methods and Automaton-Based ICS Detection.....	16
2.2 Faithfulness Research.....	18
2.3 Runtime-Verification Theory.....	19
2.4 The European Union AI Act and Safety-Critical AI.....	20
2.5 Explainability and Operator Trust.....	21
2.6 Comparison of Related Approaches.....	21
2.7 Position of This Thesis.....	22
3 Methodology.....	24
3.1 Research Philosophy and Design.....	24
3.2 Three-Layer Architecture.....	25
3.3 Formal-Bound Schema.....	26
3.3.1 Proxy Event Types.....	28
3.4 Reasoning-Violation Taxonomy.....	28
3.5 Verification Mechanism.....	30
3.5.1 Structured Claim Extraction.....	30
3.5.2 Bound Comparison.....	31
3.5.3 Violation Classification.....	31
3.5.4 Sentence-Level Coherence (V5).....	31
3.5.5 A Worked Example.....	32

3.5.6 The Verification Dashboard.....	33
3.6 Datasets	33
3.7 Proxy Event-Type Distribution.....	35
3.8 Language Models.....	35
3.9 Prompt Design	36
3.10 Sampling and Evaluation Protocol.....	37
3.10.1 Evaluation Metrics	38
3.11 Manual Annotation	39
3.12 Tracking, Reproducibility, and Data Provenance	41
3.13 Research Ethics.....	41
4 Results.....	42
4.1 Violation Patterns per Model	43
4.2 Detector Performance on the Annotated Set.....	45
4.3 Violation Patterns per Event Type	48
4.4 Statistical Association.....	49
4.5 Latency and Operational Cost.....	50
4.6 Qualitative Examples	51
4.7 Summary of Findings.....	52
5 Discussion	53
5.1 Empirical Implications.....	53
5.2 Threats to Validity	55
5.3 Interpreting the Low Agreement.....	56
5.3.1 Comparison with Related Work.....	57
5.4 Operational Deployment Guidelines.....	58
5.5 Mapping to the European Union AI Act.....	59
5.6 Limitations and Future Work.....	61
6 Conclusion	62
6.1 Summary of Contributions.....	62
6.2 Answering the Research Questions	62
6.3 Future Work	64
References.....	66
Appendix A Reasoning-Violation Taxonomy in Full Detail.....	68
A.1 V1 — Fabricated Reasoning	68
A.2 V2 — Contradicted Reasoning	68
A.3 V3 — Over-Generalised Reasoning	68
A.4 V4 — Under-Specified Reasoning	68
A.5 V5 — Incoherent Reasoning.....	69

Appendix B Prompt Specifications.....	70
B.1 System Prompt	70
B.2 User Prompt Template	70
B.3 Generation Parameters	70
Appendix C Reproducibility Notes.....	71
C.1 Software Versions	71
C.2 Reproduction Commands	71
C.3 MLflow Experiments	71
Appendix D Generative-AI Tool Disclosure	72

List of Figures

Figure 4.1 V1–V5 violation rates per model

Figure 4.2 Clean-response rate per model

Figure 4.3 Confusion matrix, human label vs. ReasonGuard verdict

List of Tables

Table 2.1 ReasonGuard positioned against related faithfulness-evaluation approaches

Table 3.1 Preprocessing row counts, raw CSV to formal bound

Table 3.2 Column dictionary: raw CSV field to formal bound feature

Table 3.3 Proxy event-type distribution over the 20,000 IEC-104 bounds

Table 3.4 Severity mapping from violation set to severity level

Table 4.1 Per-model violation summary

Table 4.2 Per-class detector performance on the 200-pair annotated set

Table 4.3 Per-event-type clean rate for each model

Table 4.4 Chi-square tests of independence

Table 4.5 Per-prompt latency per local model

List of Abbreviations

AI	Artificial Intelligence
APA	American Psychological Association (citation style)
ASDU	Application Service Data Unit (IEC-104 payload structure)
COT	Cause of Transmission (IEC-104 field)
CPU	Central Processing Unit
CSV	Comma-Separated Values
DNP3	Distributed Network Protocol 3
DPA	Deterministic Probabilistic Automaton
F1	Harmonic mean of precision and recall
ICS	Industrial Control System
IEC-104	IEC 60870-5-104 tele-control protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
LTL	Linear Temporal Logic
MLflow	Machine-Learning experiment-tracking platform
MMS	Manufacturing Message Specification
NER	Named-Entity Recognition
NES@FIT	Network and Embedded Systems group, FIT VUT Brno
QMS	Quality Management System
RQ	Research Question
RV	Runtime Verification
V1-V5	The five ReasonGuard reasoning-violation classes
VUT	Brno University of Technology

1 Introduction

1.1 Background and Motivation

Trustworthy explanation. Imagine an alarm sounding in the control centre of a power station: something has gone wrong and action is required. If the station is compromised, immediate and effective mitigation is crucial to minimise damage. Under pressure to act, a control room operator turns to the details of the alarm to determine the nature of the issue and what steps should be taken to address it.

Assume the alarm is raised by a formal detector that monitors traffic on the private communication network within the station, alerting on messages that violate the formal specification of the communication protocol used between equipment. We take as a running example the IEC 60870-5-104 (IEC-104) protocol, used to convey data from substations in an electricity network to a control centre, though similar ideas apply to a range of industrial control system (ICS) protocols, for example, the Modbus protocol used in factory automation, or the DNP3 protocol used to manage utilities. Operators may also benefit from explanations for alerts from detectors that identify anomalies in sensor readings from the station; this is left for future work.

Formal detectors provide mathematical guarantees about their verdicts; we can be certain that when such a detector raises an alarm, the traffic really does violate the specification. However, it may take an engineer some time to understand why an anomaly was flagged by inspecting a trace of the offending messages. Moreover, they must decide whether the anomaly is malicious or benign; some protocol violations are harmless, others may be indicative of an attack. As such, it would be useful for the detector to provide a more actionable explanation of its verdict, summarising the issue and providing some indication as to its cause. We call explanations faithful if they provide a true and complete summary of a verdict. A faithful explanation can help an analyst quickly grasp what has gone wrong and take effective action; a system that faithfully explains its verdicts is trustworthy.

Why not let an LLM write the explanation? Large language models (LLMs) can be prompted with a detector's verdict to generate an explanation without training on domain-specific data, and produce fluent and readable summaries that can reduce the cognitive load on operators. However, LLMs may hallucinate: generating text that is unsupported by the context they were prompted with. This can introduce false or misleading information into the explanation. We would like explanations generated by LLMs to be faithful to the detector's verdict; however,

LLMs can contradict the input or omit crucial details. A control room operator presented with an unfaithful explanation may make the wrong judgement as to whether there is a problem, wasting valuable time to resolve a non-existent issue or failing to identify and respond to an attack. Trustworthy explanation is therefore not merely desirable; it is a safety-critical concern.

ICS such as electricity grids, water management, or factories control physical systems; interference with them has tangible effects in the real world. Their communication protocols were standardised decades ago and lack security measures such as authentication and integrity checks. Unlike software, the machinery they control has lifespans measured in decades, so upgrading them in response to emerging threats is impractical, leaving them vulnerable to attack. Indeed, there have been a number of high-profile attacks against energy and other critical infrastructure over the last decade (Liang et al., 2017). Attackers target the ICS protocols used between physical equipment to damaging effect; time is of the essence between an attacker sending malicious traffic and being able to bring down, say, a substation, or worse. Operators need tools that provide accurate and actionable insight into any detected anomalies as soon as possible to mitigate against this.

Previous work has focused on using metrics to evaluate LLM-generated explanations offline: scoring them for faithfulness to detector verdicts and their utility to engineers and operators. Such metrics allow LLM authors and operators to compare models on a range of examples and choose one that performs well on average. However, a safety-critical application requires more: explanations are consumed by operators one at a time and decisions are made based on the content of each. Metrics that score explanation quality offline offer no guarantee about individual outputs. We seek to ensure explanations are trustworthy before they are shown to an operator, not merely select the LLM that errs least often. This calls for a notion of verification of explanations online, after they are generated by an LLM and before they are consumed. This may allow an unreliable explanation to be withheld, flagged for review, or regenerated with additional prompting.

We propose runtime verification to reconcile the fluency of LLM-generated text and the rigour demanded in a safety-critical application. Fluency is what makes explanation effective: it is what allows busy and time-strapped control room operators to quickly understand alerts; unfortunately, it is also what makes hallucination dangerous, as LLMs can convincingly assert falsehoods. Natural language is expressive, allowing LLM-generated explanations to go wrong

in subtle and pernicious ways. Runtime verification provides a framework to reason about such systems; it allows us to identify and address these issues.

1.2 Problem Statement

A critical aspect of these pipelines that seems to be missing is the ability to determine whether the LLM's explanation is consistent with the verdicts of the underlying formal detectors.

The goal of the formal-methods side of the problem is to determine whether some mathematical model of a protocol is correct or incorrect. On the other hand, the goal of the natural-language side is to generate text that is as fluent and helpful as possible. In neither case is there a notion of quality or correctness with respect to whether the explanation is faithful to the verdicts generated by the detector. There is also nothing within the pipeline that prevents the LLM from making assertions that go beyond those made by the detector.

Recent work has been done to quantify factual consistency of generated text given some premise (Honovich et al., 2022). One line of research proposes breaking up the premise and generated text into atomic facts, then quantifying the extent to which each atomic fact in the generated text is supported by the premise. While this is promising work, we believe such metrics are helpful to compare generated texts offline, such as to choose between LLMs, but less so in terms of quantifying the safety of an explanation for immediate operational use. These metrics quantify how likely it is, on average, that the explanation is consistent with the premise. On the other hand, control room operators need explanations that are actionable. It is not always clear whether an explanation can be trusted to make decisions. Also, in contrast to formal verification, using natural-language premises reintroduces a level of ambiguity.

Currently, there is nothing within the pipeline that would indicate whether the explanation makes assertions that go beyond those made by the verdict. Safety-critical systems need a notion of correctness with respect to the bounded nature of an explanation. Operators need explanations that stay true to the fields and types within a verdict. Such systems should be able to alert users to any violations of these bounds, in a taxonomy that is actionable for the end user.

The trend towards coupling formal detectors with a general-purpose LLM seems promising. Such LLMs are highly fluent and cheap to run on operational hardware. The novelty of such systems could be hiding deeper integration issues that may become problematic to address down the line. It seems easier and more impactful to identify such issues now, while they are

on the rise. It seems this issue has fallen through the cracks between the NLP and formal-methods communities, motivating this thesis to explore this idea further and propose a solution.

1.3 Research Questions

Can formally verified system outputs be utilised as runtime verification bounds to detect and classify violations of AI reasoning? Is this approach actionable and reliable for safety-critical use cases?

This central research question can be broken down into the following sub-questions that are answered throughout the different chapters of this thesis:

(RQ1) How can detection results of formal, automaton-based violation detection be structured to create formal verification bounds that can be utilised during runtime verification?

(RQ2) How can violations of AI reasoning with respect to formally specified runtime verification bounds be categorised, and what does such a taxonomy look like when considering AI-based anomaly-explanation tasks for ICSs?

(RQ3) How accurately and reliably can AI reasoning violations with respect to formally verified system outputs be detected with ReasonGuard for different event types in ICSs across various local LLMs?

(RQ4) Under which circumstances is AI reasoning most likely to violate formally verified bounds, and how do factors such as LLM size or the complexity of ICS events influence AI reasoning violations?

(RQ5) What guidelines for a safe deployment of LLM-based explanation systems follow from our results, and how can these guidelines support the requirements for transparency and quality management from, for example, the EU AI Act?

1.4 Significance of the Study

This work makes use of language models in combination with a formally verified detector for producing explanations of its verdict. However, in the case of a mismatch between verdict and explanation, only the former is subject to safety guarantees, although it is the explanation the operator is exposed to and acts upon.

Thus, verifying explanations at runtime with respect to formal guarantees, as proposed in this work, raises awareness of these kinds of errors by turning silent errors into explicit, categorised

ones, and by withholding or flagging faulty explanations before the operator takes action upon them. Further, it offers an avenue to assess and choose between candidate models with respect to a concrete, safety-critical task.

Moreover, such proofs-of-concept contribute to compliance with the quality management and transparency obligations introduced by the European Union AI Act for AI applications in the context of critical infrastructure (European Parliament & Council, 2024).

For research into explanation faithfulness, this work contributes a novel perspective by considering faithfulness in the sense of verification at runtime with respect to mathematical guarantees, instead of the common research direction of judging faithfulness a priori or a posteriori with respect to an explanatory document.

While this work's concrete, rule-based approach has significant strengths, it also exhibits notable limitations; these are discussed alongside the results in order to contribute an honest picture of what can be achieved using this approach.

1.5 Scope and Delimitations

All results, examples, and other artefacts using real-world traffic were produced on a dataset of smart-grid traffic conforming to the IEC-104 protocol collected on one of the testbeds of the NES@FIT group at Brno University of Technology. While the overall solution is not limited to the IEC-104 protocol, no claim is made that it can be applied as is to any ICS protocol, for example, Modbus, DNP3, S7 or similar, or even to domains outside ICS without further changes or verification.

Similarly, while the solution is specified in terms of automata, the actual verification is done on a proxy classifier instead of the official automaton definition of the protocol. Results depend on the particular proxy used and are as such appropriately labelled throughout the rest of this thesis. The solution was tested on a number of LLMs, including three local and two cloud-based, but only on a single quantisation as an example. No claim is made about how well it can generalise to other LLM/quantisation combinations.

Lastly, the proposed verifier was evaluated in terms of ground truth provided by humans. No interactive, operator-in-the-loop experiment was conducted and no claim is made about how the proposed solution would affect the behaviour of control-room operators if it were deployed. These limitations will be revisited in Chapter 5 as threats to validity and opportunities for future research.

1.6 Contributions

Four main contributions are presented in this thesis.

- **The ReasonGuard framework for verifying AI reasoning.** This thesis presents ReasonGuard, to the best of the author's knowledge, the first runtime-verification framework that uses the output of a formally verified system as the bound to verify AI-generated explanations of this output at runtime. Note that the bound is not the official NES@FIT automaton; it is a proxy classifier which is a component of ReasonGuard. The whole structure and verification are explained in Chapter 3.
- **A taxonomy of violations of reasoning.** Based on literature from the fields of faithfulness (Maynez et al., 2020) and runtime-verification (Leucker & Schallhart, 2009), a taxonomy of five distinct classes of violations (labelled V1 to V5), namely reasoning that is fabricated, contradicted, over-generalised, under-specified and incoherent, is proposed. For each of the violation classes, its formal definition, the means of detecting it, and its empirical frequency of occurrence are given.
- **Evaluation of ReasonGuard.** The framework is evaluated using an industrial control system dataset for the IEC-104 communication protocol created by the NES@FIT group at Brno University of Technology (Matoušek et al., 2022), and violations are measured for three local and two cloud language models and four observed ICS event types. Violation patterns observed between language models and ICS event types are discussed and verified for statistical significance.
- **Guidelines for LLM deployment.** The results of the empirical study are generalised into guidelines for entities interested in deploying LLM-based explanation systems when a formally verified component is used. The guidelines are compared to the transparency and quality management requirements for high-risk AI systems specified in Articles 13 and 17 of the European Union AI Act (European Parliament & Council, 2024).

1.7 Thesis Structure

- The "red line" continues as follows. The next chapter introduces a literature review, discussing previous work on the four aspects: formal methods for automata-based detection of ICS, faithfulness and hallucination research on large language models, theory of runtime-verification, and accountability/transparency in the context of the EU AI Act.
- This is followed by the methodology chapter, Chapter 3, which introduces the central contribution of the thesis: ReasonGuard, the taxonomy of reasoning-violation types (V1–

V5), a three-layer architecture, a bound schema for reasoning, the proposed verification mechanism, and details about the models, prompts, datasets, evaluation protocol, and research ethics followed in this research.

- Then, results and discussion in Chapters 4 and 5 present the results, discuss them along with potential threats to the validity of the results, present deployment guidelines derived from the findings of this work, and relate them to the EU AI Act. Chapter 6 summarizes the contributions of this thesis, answers the research questions and presents possible future research.

2 Literature Review

This thesis builds upon four different areas of research. These areas of research are formal methods, faithfulness research, runtime verification and the EU AI Act.

Currently, none of these research areas on their own verify the quality of an explanation with respect to a verdict of a highly reliable method like the ones found in the field of formal methods. However, taken together, the contributions of these areas of research provide a way of verifying Natural Language Explanations with respect to such verdicts on runtime. Such verification can contribute to auditable compliance with the EU AI Act, especially concerning the requirement for quality management systems and transparency.

ReasonGuard puts forward a new approach that utilizes these fields of research by considering the verdict of a method in the field of formal methods as a (machine-readable) specification that a given explanation has to satisfy.

2.1 Formal Methods and Automaton-Based ICS Detection

An important topic related to this thesis is model checking (Clarke et al., 2001) – a formal verification method that checks if a given (finite-state) model of a system satisfies a specification describing its temporal properties. In particular, the specification must hold for all possible executions of the model. Such a model should not be too large, as otherwise all possible executions cannot be analysed, but sufficiently expressive to capture the specified properties of interest.

This approach can be used also for anomaly detection – the specification describes legitimate behaviour of the system, and anomaly detection then consists of checking if the specification holds. This way of anomaly detection brings an important difference compared to others – the

result of anomaly detection is a proof that the given situation is/isn't an anomaly, not merely its score based on some heuristics. Naturally, such proof is only valid with regard to the given model of the system. This approach has a big advantage compared to large language models, which cannot be bounded and modelled in a tractable way. Their behaviour can differ based on the prompt, and moreover, they are opaque to some degree. Thus, it is not possible to generate such proofs of their properties for all possible executions. This impossibility motivates runtime verification approaches presented in Section 2.3.

As a concrete example of anomaly detection with automata in the context of ICS, Havlena et al. (2023) present a technique for anomaly detection in ICS communication protocols based on deterministic probabilistic automata learned from legitimate communication. Their approach is instantiated for the IEC-104 protocol and evaluated on captured ICS communication. An important aspect of this approach is that it takes into account not only superficial characteristics of the communication protocol, but also the meaning of messages, which can decrease the number of false positives. Afterwards, if the learned deterministic probabilistic automaton encounters a message it did not expect, an anomaly is reported. The output of their anomaly detection approach is a verdict containing more information about the anomaly – its type, devices it involves, timestamp of the problematic message, or an easy-to-understand explanation of the reason for the verdict.

Their approach has been verified using captured IEC-104 communication and compared to related work – anomalies are better detected using their approach. They also evaluate their technique on MMS protocol communication to show its applicability also to other ICS communication protocols. Note that the detector of Havlena et al. (2023) gives a specification of what should not be included in the explanation that ReasonGuard verifies. Unfortunately, we did not have access to their implementation, namely the NES@FIT schema and automaton, for this thesis's needs, so the detector itself was not executed. Instead, a proxy classifier that is part of ReasonGuard (see its design in Section 3.3) was used to obtain the bound based on the labels from the dataset. This work comes from the NES@FIT group of the Faculty of Information Technology at Brno University of Technology. Another approach can be found in the first supervisor's previous work (de Silva et al., 2013), where reachability analysis using temporal logic is used and its result is nicely formatted to be used by another component of the whole system, similarly to ReasonGuard which obtains its input from another component using formal methods.

2.2 Faithfulness Research

Faithfulness research has long distinguished factuality with respect to the world from faithfulness to source(s) (Maynez et al., 2020), a distinction that persists today. Maynez et al. (2020) provide an early benchmark in this domain. Given the rapid development of LLMs since then, it is necessary to get an updated picture of this domain. Huang et al. (2025) give a comprehensive survey on recent advances in this field, presenting a modern taxonomy as well as a summary of the discussion on faithfulness and hallucination in neural generation. For example, FActScore (Min et al., 2023) highlights the utility of analysing faithfulness with respect to atomic facts in a natural-language generation (NLG) system's output, as opposed to analysing the output holistically. The TRUE benchmark (Honovich et al., 2022) unifies datasets for faithfulness evaluation, providing insights into the strengths and weaknesses of different faithfulness metrics. By applying existing metrics across eleven benchmarks, Honovich et al. (2022) find that natural-language inference (NLI) and question generation/answering (QG/A) metrics perform best and complement one another. Their results show that learned metrics perform best overall. ReasonGuard departs from this body of work in several ways. First, ReasonGuard provides a deterministic and auditable way of checking faithfulness, which is desirable for safety-critical domains. Second, although limited in scope, checking against a schema offers a structured, actionable, and interpretable way of assessing faithfulness: ReasonGuard returns which of a limited number of violation types occurred, as opposed to an aggregated faithfulness score. Third, this thesis considers faithfulness to formal-methods verdicts, an area of limited prior work.

The detector emits a structured verdict describing the offending behaviour together with relevant message classes, addresses, communication direction, and other protocol-specific information. A common way of using such a detector would be to feed this verdict, or part of it, into a large language model (LLM) to produce a human-understandable description of the anomaly. However, such a model would not be verified and would not formally relate to the rest of the pipeline.

This is precisely the gap this thesis addresses. On one hand, there is a lot of previous research on anomaly detection for industrial control systems based on formal models (see Section 2.1), so we can count on solid foundations to get verdicts we can be certain about. On the other hand, it has been shown that language models suffer from hallucinations (Maynez et al., 2020); this phenomenon, in combination with the terse and highly structured nature of our input, makes LLM hallucinations all but certain (see Chapter 4). Moreover, hallucinations are written in a

fully formed language just like any other model output, often sounding very confident and authoritative. As such, it is important to check whether the final description actually corresponds to the verdict produced by the model.

2.3 Runtime-Verification Theory

Leucker and Schallhart (2009) provide a common account of the goals and methods of RV. Whereas model-checking and related techniques attempt to demonstrate that all executions of a program satisfy a given property, RV asks if a single execution satisfies the property in question, and seeks to answer this question while the program is running. RV achieves this with monitors, programs that can be synthesized from formal specifications, typically written in a formalism akin to linear temporal logic, and produce verdicts as they consume inputs. By giving up the goal of ensuring that all executions satisfy the property of interest, RV gains significant tractability and applicability over model-checking, making RV well-suited to monitor systems that are large, opaque, or otherwise resistant to ex ante verification.

ReasonGuard fits neatly into this picture. The execution under inspection is the AI-generated explanation. The specification is the formal automaton produced by ReasonGuard, and the monitor is the verifier which consumes the explanation and produces a verdict. The property of interest is the faithfulness of the reasoning to the verdict, meaning, roughly, that the explanation does not assert facts which contradict or are unsupported by the verdict. As with other approaches to RV, ReasonGuard can provide evidence about whether to allow the monitored system into production, except that this evidence is available immediately, not merely for ex-post reflection.

First, like classical RV, ReasonGuard is concerned with safety properties, meaning, roughly, properties that assert that some state that ought never occur does not. Safety properties are distinguishable from other kinds of properties because violations of safety properties are refutable: a finite input suffices to determine whether the input does not satisfy the property. Other kinds of properties, such as so-called liveness properties, cannot be conclusively verified by consuming finite inputs. Faithfulness is a safety property: an explanation may make an unlimited number of true assertions without thereby satisfying faithfulness, but only one false assertion is necessary to refute it.

Second, whereas monitors in classical RV receive inputs as they evolve and must thus update their verdicts accordingly, ReasonGuard's verifier receives complete texts and processes them in a single pass.

Third, the specification in classical RV is a description of the desirable behavior of a system under observation, and the monitor must be kept independent of the system it monitors. Chapter 3 introduces a schema for bounds which enables ReasonGuard to reflect on AI-generated explanations, a capability which can be thought of as monitoring. This capability can be brought into alignment with classical RV through the introduction of a new kind of bound which rejects all possible assertions unless they are explicitly permitted. With a fail-closed bound like this one, the monitor's independence from the generative system under observation can be preserved. Table 2.1 summarizes these points of similarity and distinction.

Leucker & Schallhart (2009) discuss the basics of the theory of runtime verification. More recently, Taleb et al. (2023) present an updated survey on runtime verification, including discussions of uncertain and incomplete execution traces. ReasonGuard has several challenges with respect to state abstraction and uncertainty via its use of proxies; these challenges map to the theoretical framing in the aforementioned paper.

2.4 The European Union AI Act and Safety-Critical AI

Regulation (EU) 2024/1689 (European Parliament & Council, 2024), known as the European Union Artificial Intelligence Act or AI Act, outlines requirements for AI systems sold into the European Union. Amongst other requirements, the AI Act distinguishes high-risk systems which require stronger guarantees. Annex III Point 2 of the AI Act classifies AI safety components in critical digital infrastructure, or in the management and operation of essential services in digital infrastructure, water, gas, central heating, or electricity as high-risk systems.

In order to be permitted into production, high-risk AI systems must satisfy several requirements. Amongst these are Article 13, which calls for sufficient transparency to enable intended recipients to be able to interpret outputs and use the system appropriately, and Article 17, which calls for a documented, effective quality-management system to oversee AI system development and deployment. Such a quality-management system may include policies and procedures for testing, record-keeping, monitoring, evaluating, and taking corrective action.

Translating safety-critical AI systems' outputs into natural language explanations for use by control-room operators squarely intersects with these requirements. Not only does it suggest a way of satisfying Article 13 by generating explanations, it also brings the content of such explanations into scope. As discussed in Section 2.5, operators suffer from alert fatigue and are likely to trust fluent, confident explanations. By monitoring faithfulness in such situations, ReasonGuard both prevents AI-generated explanations from misleading operators and produces evidence which can contribute toward satisfying Articles 13 and 17 of the AI Act.

While ReasonGuard does not aim to satisfy Articles 13 and 17 in full, the EU AI Act supplies the governance obligation that makes such a check necessary rather than merely useful in safety-critical ICSs.

It is important to acknowledge the limitations of the runtime-verification analogy: a classification task is not identical to a control task, and automaton acceptance is not identical to the more graded notion of faithfulness in explanation research. Nonetheless, the analogy captures the core formalization introduced by this thesis: explanations can be treated as observable behaviour that is checked against a specification during generation. The resulting verdicts support auditing, selective presentation, and safety-critical monitoring. This way of thinking will be picked up again in Chapter 3, where we develop the detailed approach used in the verification part of our proposed solution.

2.5 Explainability and Operator Trust

An important stakeholder group is control room operators. It emerges that operators are subject to high volumes of alerts, which encourages low-effort processing. This rewards whatever presentation is easiest to accept, a fluent narrative most of all. On the other hand, automated, fluent, confident explanations may be trusted more than warranted (Parasuraman & Riley, 1997). This makes runtime monitoring of faithfulness particularly important: misleading but fluent explanations actively suppress more cautious reasoning. Moreover, monitoring results must be presented to operators in a way that supports trust calibration under time pressure, for example, categorical verdicts, not just opaque confidence scores.

2.6 Comparison of Related Approaches

Table 2.1 positions ReasonGuard relative to the three lines of related work discussed earlier concerning evaluation of faithfulness. It summarises what is being compared, when, and the nature of the comparison results.

Table 2.1 ReasonGuard positioned against related faithfulness-evaluation approaches.

Approach	Compared against	Timing	Output
Maynez et al. (2020)	Source document	Offline	Human / entailment judgement
FactScore (Min et al., 2023)	Knowledge base	Offline	Per-claim precision score
TRUE (Honovich et al., 2022)	Source document	Offline	Consistency score
ReasonGuard (this thesis)	Formal verified bound	Runtime	V1–V5 category + severity

2.7 Position of This Thesis

ReasonGuard synthesises four previously separated threads of research. These are:

- automaton-based methods for formally recognising and reacting to particular structures within text
- the line of faithfulness evaluation based on decomposing explanations into individual claims
- the theory of runtime verification described by Leucker and Schallhart (2009) and others
- recent calls for trustworthy and verifiable AI from the governance community

This combination of requirements leads to a novel approach to faithfulness evaluation, characterised as follows. It takes place: (i) at generation time, on the completed explanation before it reaches the operator, not ex-post; (ii) in the context of a broader explainability process; and (iii) fail-closed, not fail-open. Furthermore, it is motivated by a novel application of faithfulness in the context of the industrial control systems (ICS) domain, where explanations are monitored as they are provided to a human. Overall, to the best of our knowledge, ReasonGuard represents a new application of RV to AI-generated text and a new approach to faithfulness evaluation, introducing a formally-based runtime guarantee between AI and human. We can claim ReasonGuard as the first application of RV to LLM interactions, although it is important to clarify this statement as there is existing work on the topic. RV4Rasa (Ferrando et al., 2023) applies RV to chatbot systems. The primary difference between ReasonGuard and RV4Rasa is the properties being monitored. While RV4Rasa monitors if the conversation adheres to a conversation protocol, that is, a set of predefined dialogue flows,

ReasonGuard monitors the faithfulness of the text with respect to the verdict. The next chapter details our proposed RV approach.

3 Methodology

This chapter presents the ReasonGuard methodology in full. It first states the research philosophy and design (Section 3.1); it then describes the three-layer architecture (3.2), the formal-bound schema (3.3), the reasoning-violation taxonomy (3.4), and the verification mechanism (3.5). The remainder specifies the empirical apparatus: the datasets (3.6), the proxy event-type distribution (3.7), the language models (3.8), the prompt design (3.9), the sampling and evaluation protocol with the metric definitions (3.10), the manual-annotation procedure (3.11), the reproducibility arrangements (3.12), and the research ethics (3.13). The chapter answers RQ1 and RQ2 in full and lays the groundwork for RQ3 to RQ5.

3.1 Research Philosophy and Design

This research assumes a realist ontology: the bound is the objective truth embodied by the formal detector. From this, an objectivist epistemology naturally follows: the meaning of the claims lies in their relation to the bound, not in their interpretation by readers. In turn, a positivist perspective leads the research, since the goal is to evaluate, predict, and measure. The research is conducted from the perspective of an applied data scientist, and is hence situated in a pragmatic research tradition, where techniques are chosen to answer the given questions.

Methodologically, the research follows a dataset-analysis design. The research starts from a theory about formal reasoning and how it can serve as a constraint for generated explanations at runtime, then defines a taxonomy of reasoning violation classes as well as a procedure to extract reasoning patterns and violations given a formal bound. The research questions RQ1 and RQ2 can be answered based on this deductive approach, while also laying the foundation for the remaining RQs (RQ3–RQ5).

Empirically, the study includes a human annotation to establish a reference, and uses both qualitative and quantitative measures for evaluation. Quantitative measures include violation tallies and class-wise evaluation, and, if applicable, tests of association. Human annotation provides a reference and enables qualitative analysis of explanation quality and faithfulness. The goal is to explore and understand the characteristics of explanations, their reasoning, and faithfulness.

3.2 Three-Layer Architecture

This section describes the overall structure of the proposed runtime verification framework ReasonGuard, followed by a schema for formal bounds. The proposed taxonomy of violations and reasoning patterns is then introduced, followed by the method of reasoning pattern extraction used for verifying generated explanations.

ReasonGuard comprises three loosely-coupled layers:

- The formal layer receives raw IEC-104 frames as input and detects safety violations, thereby acting as the formal verifier (instantiated by a proxy classifier in the present evaluation; see Section 3.3). The output is a so-called formal bound that formalises the detected safety violation, its cause, and remediation, see Section 3.3.
- The AI layer receives formal bounds as input and generates explanations. The inputs are structured as prompts (system and user prompts) which are passed on to an LLM, see Section 3.9. The output is a natural-language explanation that attempts to justify the verdict of the formal verifier.
- The verification layer receives formal bounds and explanations as input and checks whether the latter are faithful to the former. The method involves extracting reasoning patterns from explanations and comparing them to the formal bound. The output is a set of violations with a severity rating, see Section 3.5.

The loose-coupling of these three layers is an important aspect of ReasonGuard. In particular, it is important that the AI layer is separated from the rest of the pipeline, as this emphasises the role of AI-generated explanations as nothing more than a natural-language interface. The final verdict is always embodied by the formal layer. The goal is that explanations provide operators with more detail and nuance that can be used for decision-making in a safety-critical setting.

A central concept of ReasonGuard is the notion of reasoning. For verification of explanations, reasoning is defined as claims about causes and consequences. Unfaithful reasoning is divided into five violation classes:

V1 — Fabricated reasoning.

V2 — Contradicted reasoning.

V3 — Over-generalised reasoning.

V4 — Under-specified reasoning.

V5 — Incoherent reasoning.

Since this work deals with safety properties, the default class is V4, meaning explanations are not to be approved unless it has been reasoned why they should be. The classes are detailed further in Section 3.5.

The schema for formal bounds has been iteratively improved during the project. At this writing, the schema is provisional and has been tagged `_proxy_until_official_schema` in the code. An official schema will be developed as part of future work.

Pattern extraction is done by mapping claims to domain-specific reasoning patterns. These reasoning patterns are domain-specific and can be defined in advance to some degree. This is a strength of the approach since safety-critical applications favour deterministic approaches that allow for reasoning about behaviour, albeit at the cost of recall. This can be contrasted with statistical approaches that would automatically learn reasoning patterns from data.

Each stage produces a file (bounds set, prompts set, responses set, verification report), enabling inspection, re-runs, and replacement of artefacts at any stage. This also allows all reported statistics to have provenance, that is, it is always clear which file a statistic was calculated on. This practice will be referenced in Section 3.12 when claiming reproducibility, as well as Chapter 5 when discussing verification artefact auditing.

3.3 Formal-Bound Schema

This subsection presents a concise overview of the formal-bound JSON format. A complete example bound is shown in Listing 3.1; the violation logic these constraints map to is given in Appendix A.

A bound consists of metadata, observable features (such as protocol, ASDU fields), claims that are allowed, required, or forbidden to appear in an explanation, and flags indicating whether each constraint is machine-checkable (that is, by the NES@FIT automaton). For example, the assertion that communication is IEC-104 is machine-checkable and may appear in explanations of events labelled as any of the five classes. This constraint can be enforced by the automaton. Conversely, the requirement that the explanation must not make assertions about mitigation options is neither machine-checkable nor specific to any event. Nevertheless, both types of constraints can be represented in the formal-bound schema.

In the absence of the automaton, all flags in the `machine_constraints` object default to false, meaning no assertion is validated by formal methods and the system operates in proxy mode.

This design decision was made because it is more sensible for a safety property to fail closed, that is, defaulting to reject (in this case withholding approval of the explanation) unless proven otherwise. Once assertions are confirmed by the automaton the flag would be set to true and the `formal_bound_version` would be increased.

```
{
  "event_id": "evt_000001",
  "dataset": "eon_iec",
  "event_granularity": "atomic_row",
  "timestamp_ms": 1654646400041,
  "formal_bound_version": "2.1",
  "schema_status": "proxy_until_official_automaton_schema",
  "protocol_hint": "iec104",
  "communication_direction": "client_to_server",
  "frame_type": "s_or_u_format_control",
  "formal_class": "IEC104_CONTROL_FRAME_OBSERVATION",
  "formal_class_status": "proxy_not_ground_truth_attack_label",
  "features": { "source_id": 1, "destination_id": 2, "bytes": 46,
    "pkt_length": 4, "srcport": 57528, "dstport": 2404,
    "asdu_address": 0, "asdu_cot": 0, "asdu_items": 0,
    "asdu_type": 0, "frame_fmt": 1 },
  "allowed_claims": [ ten atomic statements ],
  "required_claims": [ five must-mention statements ],
  "forbidden_claims": [ eight must-not-mention statements ],
  "machine_constraints": {
    "attack_claim_allowed": false, "malicious_claim_allowed": false,
    "mitigation_claim_allowed": false, "causal_claim_allowed": false,
    "normality_claim_allowed": false, "scope_expansion_allowed": false },
  "proxy_event_type": "topology_change_proxy_until_official_schema"
}
```

Listing 3.1. Canonical formal bound (proxy schema version 2.1), produced by the formal-bound builder for one atomic IEC-104 row.

For conciseness the metadata and `machine_constraints` objects have been omitted. The important items to notice in the `allowed_claims` object are those that can be derived from the observable features of a given event, such as protocol, direction, source/destination, ASDU fields. The `required_claims` object specifies assertions that should be included for a satisfactory explanation. Aside from the proxy disclaimer, most are concerned with inclusion of precise values such as the ASDU fields and source/destination ids and ports. The `forbidden_claims` object specifies assertions that may not appear under any circumstances. Many of these items are difficult to validate against the event (for example, assertions about mitigation options), nevertheless it makes sense to prohibit them in order to focus explanations on the question of what happened. These three objects directly relate to violations in the taxonomy presented in the next subsection. Forbidden assertions result in V1 or V2 violations; required assertions result in V3 or V4 violations.

3.3.1 Proxy Event Types

The thesis exposé listed five types of events that could be encountered in ICS: normal behaviour (Modbus polling), Modbus function-code violation, network scan, Modbus replay attack and network topology change. In the absence of the official classification from NES@FIT the thesis implements a proxy system that can classify events into one of these categories according to simple, transparent rules applied to observable event features. All categories have the suffix `_proxy_until_official_schema` to make it easy to tell the difference between proxy and official labels, and to avoid confusion with ground-truth attacks.

3.4 Reasoning-Violation Taxonomy

This is a central contribution of the thesis and is discussed further in Chapter 4 where it is shown that the taxonomy provides satisfactory answers to RQ2. Logic is presented in Appendix A. This section provides a summary of violations with illustrative examples.

- V1 — Fabricated reasoning. The explanation makes assertions that are not backed up by the bound (for example, asserting an attack that was not labelled by the classifier).
- V2 — Contradicted reasoning. The explanation makes assertions that go against those provided by the bound. An example of this is an explanation that asserts a server is sending messages to a client when in fact it is the other way around.
- V3 — Over-generalised reasoning. The explanation makes assertions that are supported by the bound, however omits important details (for example, correctly labelling the event as a network scan but failing to provide the ids of the source/destination entities).
- V4 — Under-specified reasoning. The explanation fails to make assertions required by the bound (for example, failing to label an attack when one has been labelled).
- V5 — Incoherent reasoning. The explanation is internally self-contradictory, independent of the bound (for example, describing the same event as both normal and malicious).

From a philosophical point of view, we subscribe to the tenets of positivism and realism. The former manifests itself through the usage of human annotations as a (subjectively interpretive) reference point and comparison. The latter manifests itself through the choice of a (subjectively realistic) dataset.

However, as we are applied data scientists, the approach is far from purely positivistic: the ultimate goal of ReasonGuard is not the formulation of laws, but the improvement of human

decisions in safety-critical scenarios. Hence, the proposed methodology should be considered pragmatic and rigorous.

ReasonGuard introduces a set of classes of violations. They are named after their effect on the explanation itself, that is, from the viewpoint of faithfulness. This is a choice of convenience, because violations can be mapped to severity levels, as shown in Table 3.4. The mapping is straightforward for V1, V2, and V5, which are always high-severity. V3 and V4 require some consideration: both report true information but may omit information. On their own, both over-generalisation (V3) and under-specification (V4) are treated as low-severity, while together they become medium-severity. They are kept as separate classes because they are distinct failure modes: over-generalising loses formally established specifics, whereas under-specifying omits required information. V5 is escalated to high-severity and requires human review, because an explanation stating an event is normal and malicious at the same time is a serious failure, even though it is unlikely to appear in practice.

The proposed taxonomy may be extended to accommodate further nuances; however, the current formulation contains all the violations which may lead operators to make different decisions. Adding new classes of violations would bring a level of granularity that does not significantly affect operators' decisions.

The five classes of violations can be aggregated into three macro-categories. Violations V1, V2 and V3 can be regarded as the AI model saying too much. In particular:

- V1 — Fabricated reasoning.
- V2 — Contradicted reasoning.
- V3 — Over-generalised reasoning.

Violation V4 represents the opposite macro-category, that is, the AI model saying too little:

- V4 — Under-specified reasoning.

Finally, violations V5 do not fit either of the two macro-categories introduced before:

- V5 — Incoherent reasoning.

The approach followed in ReasonGuard for violations' definition and taxonomy is conservative and fail-closed, that is, the system defaults to rejecting explanations unless explicit grounds for approval are presented. This is the right attitude when verifying safety properties.

Table 3.4 Severity mapping from the violation set to the severity level.

Violation set	Severity
V1, V2, or V5 present (any superset)	High
V3 and V4 together (no high-severity class)	Medium
V3 alone, or V4 alone	Low
No violation	None

3.5 Verification Mechanism

For each (formal bound, AI explanation) pair, ReasonGuard runs a three-step verification pipeline: structured claim extraction, bound comparison, and violation classification. The mechanism answers the first half of RQ3 — how the framework detects each class — while its measured reliability, in Chapter 4, answers the second.

3.5.1 Structured Claim Extraction

The first part of the process uses spaCy with a custom EntityRuler for the ICS domain. In addition to the built-in statistical named-entity-recognition, spaCy allows adding more deterministic methods such as a ruler that matches predefined patterns.

The ruler contains a number of patterns for finding specific entities of interest in this work, including protocols, the direction of communication, frame types of the IEC-104 protocol, ASDU fields, network endpoints, types of attacks, normality, mitigation, and uncertainty. The ruler is added to the beginning of the spaCy pipeline, before the default named-entity-recognition. Thus, if a domain-specific entity is found, its processing stops there. More generic entities, such as numbers, as well as any entity that is open-ended, are found by the named-entity-recognition later on in the pipeline.

The benefit of using rules to find entities is predictability of the pipeline. If the same explanation is input twice, the same claims are always extracted. In addition, domain-specific words that the statistical recognition does not necessarily recognize in unusual context are flagged as entities. In a safety-critical domain it can be advantageous to augment statistical methods with deterministic methods to get predictable results.

Therefore, the trade-off of the ruler-based extraction is that claims phrased in an unexpected way cannot be recognized. Such expressions that the pattern-matching misses are part of the reason why recall is an issue for fabricated claims and will be studied in Chapter 5.

3.5.2 Bound Comparison

This second stage determines whether each of these claims is SUPPORTED (i.e., consistent with the bound), UNSUPPORTED (not consistent with the bound, but also not contradicted by it), or CONTRADICTED by the bound. The classifier checks the direction and the references to source, destination, port, ASDU, etc., and whether terms from the sets of terms for attack, mitigation, and normality agree with the corresponding flags in the machine_constraints. It also checks if each number is compatible with the union of the set of numbers from the bound and a set of constants pertaining to the protocol (e.g., for IEC-104, the protocol number 104 and its port number 2404).

3.5.3 Violation Classification

Once each claim receives a verdict, the third step maps verdict patterns onto the V1–V5 violation types using a lightweight, rule-based classifier. This classifier has two complementary parts. First, a lexicon layer applies curated term lists, such as attack, mitigation, causal, and normality terms. This layer always runs and provides a conservative baseline. Second, a syntactic layer builds a spaCy dependency parse of the response, which helps determine whether candidate terms are truly asserted. For instance, an attack or mitigation term is marked as negated if spaCy links it to a negation in the same sentence or if the sentence contains clear schema/proxy markers ("proxy", "schema"). Lexicon-based flags are honored only when the term passes these filters. This design makes the classifier more robust than simple matching by distinguishing mentions from assertions and reducing false positives from negation or meta-discussion. If spaCy is unavailable, the classifier falls back to a restricted, allowlist-based negation heuristic using a few well-formed substring patterns. This covers the most common negation phrasings in our synthetic regressions, but may miss others — for example, more complicated negation structures that may occur in responses generated by local LLMs.

3.5.4 Sentence-Level Coherence (V5)

We use spaCy's sentence segmentation to detect V5 violations. We then check whether known conflicting terms, such as "normal" and "malicious", or "client-to-server" and "server-to-client", appear in the same sentence. We also check whether negation is present to ensure we

do not trigger for responses that indicate that traffic is neither normal nor malicious. This design choice of checking for contradictions on a sentence-wise basis, as opposed to document-wise, reduces false positives.

3.5.5 A Worked Example

Listing 3.1 shows a simple example of a bound allowing ReasonGuard to check explanations of an IEC-104 frame with source and destination identifiers 1 and 2.

Suppose an LLM produces an explanation of this frame, sent over its standard port 2404 and with all-zero ASDU fields. The following is an explanation we have crafted by hand to be faithful to the ground-truth data:

- *This is an IEC-104 control frame sent from the source with identifier 1 to the destination with identifier 2.*
- *Its ASDU fields are all zeros.*
- *The frame was observed in traffic on port 2404.*
- *Because all of the machine-checkable constraint flags are false, the frame cannot be confirmed as normal.*
- *Because the classifier is only a proxy for maliciousness, the frame also cannot be concluded to be an attack.*

ReasonGuard labels every sentence of this explanation as supported or consistent with the input; no violations occur and Severity is set to none.

Next we discuss several ways this explanation could have been worded to violate the bound, triggering ReasonGuard's detectors.

An explanation might make an attack claim not justified by the bound, for example, "This looks like a replay attack." Here, one of the claim-extractors would extract the noun phrase "replay attack" and identify it as referring to an attack. Since `attack_claim_allowed` is false and the term is not negated, ReasonGuard identifies an unsupported attack claim, and Severity becomes high because of a violation of the first rule, V1. Similarly, ReasonGuard would identify a reference to a "compromised" device as an unsupported claim of maliciousness.

The explanation could include a statement that contradicts the ground-truth data specified by the bound, for example, "Traffic goes from device 2 to device 1." ReasonGuard would identify

this as a contradiction of the direction of communication, given by the bound to be `client_to_server`, and would escalate Severity according to the second rule, V2.

The explanation might fail to make a required claim. For instance, we could remove any mention of the port number or the ASDU fields from the bullet points above. In that case, even though the explanation avoids forbidden claims, ReasonGuard would identify missing required claims and escalate Severity to low according to the fourth rule, V4.

This simple bound suffices to illustrate how ReasonGuard's detectors respond to explanations containing claims that are fabricated, contradicted by ground-truth, or under-specified relative to ground-truth. For regression-testing ReasonGuard, we have a larger dataset of synthetic data that includes several more complex bounds and their inputs, the faithful response for each, and exactly one example of each kind of violation. This provides a reliable ground-truth for unit-testing ReasonGuard.

3.5.6 The Verification Dashboard

To provide a user-friendly way to verify explanations, a dashboard is implemented as a Streamlit application which can be accessed in a browser. Here, the person verifying explanations can select a bound and its fields, view allowed, required, and forbidden claims and enter a given explanation to verify it.

After submitting an explanation, violations found by ReasonGuard are shown along with a severity indicator. By checking a box, the spaCy model can be activated, which provides insight into the three-valued verdicts for each claim. These help to associate claims and violations. The dashboard's transparency requirements from Section 2.5 and compliance with Article 13 presented in Chapter 5 are thereby addressed. The evidence shown in the dashboard allows the verifier to overrule ReasonGuard's severity if necessary. The application does not demand blind trust in ReasonGuard's severity.

The dashboard shares the same code with the evaluation pipeline. Hence, its metrics correspond to the evaluation. Its live version can be accessed via the URL given in Appendix C.

3.6 Datasets

Two datasets have been made public by NES@FIT group at the Brno University of Technology. Both of them were supposed to be used according to the exposé. The first one, titled Modbus Dataset for ICS Anomaly Detection (Ryšavý & Matoušek, 2021), contains samples of Modbus network traffic stemming from a controlled testbed. The idea was to use it

for the extraction of function-code-violation and replay-attack anomalies. However, this dataset is not used anywhere in the pipeline at the moment. This counts as one of the limitations and future works discussed in Chapter 5.

The second one, titled ICS Dataset for Smart Grid Anomaly Detection (Matoušek et al., 2022), contains samples of IEC-104 network traffic stemming from a controlled testbed emulating a smart-grid setup. It is the source of the eon-iec subset of data used by the pipeline, which serves as a proxy for the Modbus-specific anomaly types. In this sample, function-code violations appear, whereas replay attacks do not (see Section 3.7 and Table 3.3). It contains a total of 20,000 atomic rows representing IEC-104 protocol messages. Throughout its transformation from raw CSV data to a set of messages in their formal-bound representation, none of them is filtered out, removed, or otherwise lost. Hence, the final dataset gives a full image of its original, raw counterpart.

Table 3.1 Preprocessing row counts, from raw CSV to formal bound.

Preprocessing step	Count
Raw CSV rows (sample_003.csv)	20,000
Formal bounds produced	20,000
Rows dropped or filtered	0
Bounds sampled for the LLM prompt batch	100

Table 3.2 Column dictionary: each raw CSV field and the formal-bound feature it populates.

Raw CSV column	Formal-bound field
srcip	features.source_id (anonymised integer)
dstip	features.destination_id (anonymised integer)
time	timestamp_ms
bytes	features.bytes
pkt_length	features.pkt_length
srcport	features.srcport
dstport	features.dstport
asdu_address	features.asdu_address
asdu_cot	features.asdu_cot
asdu_items	features.asdu_items
asdu_type	features.asdu_type
frame_fmt	features.frame_fmt

3.7 Proxy Event-Type Distribution

Using the proxy classifier we classify the 20,000 IEC-104 bounds into four different event types, as the fifth type, replay attack, has no occurrences in our sample.

Note that events of the IEC-104 protocol are relabeled as shown in Table 3.3. More information about this can be found in Section 3.6.

This means that, for example, a label like 'normal_modbus_polling' is not related to Modbus events. They are actually IEC-104 events used as proxies for the Modbus anomalies.

Table 3.3 Proxy event-type distribution over the 20,000 IEC-104 bounds.

Proxy event type	Count
normal_modbus_polling	16,205
topology_change	3,683
network_scan	68
function_code_violation	44
replay_attack	0 (absent from sample)

Getting the function-code-violation and replay-attack types directly from the Modbus dataset is left to future work. Otherwise, the evaluation includes four of the five canonical event types, all within the proxy regime.

3.8 Language Models

For the local models, we used Ollama version 0.23.3 with the following tags: llama3.1:8b-instruct-q4_0 at ~4.7 GB, mistral:7b-instruct-v0.3-q4_0 at ~4.1 GB, and phi3:mini at ~2.2 GB. We chose 4-bit quantisation tags as a balance between quality and resource use, and ran inference on CPU. All model calls use the same sampling parameters, with a temperature of 0.1 and top-p of 0.9, so differences in outputs come from model behaviour rather than sampling settings.

For cloud models, we used OpenAI's gpt-4o-mini and Google's gemini-2.5-flash-lite. This is a change from our initial model selection, which included gemini-1.5-flash, a model which has since been retired by Google. gemini-2.5-flash-lite is Google's current offering at a similar level. Our cloud-model runner is a thin client around these APIs, and will skip the model if the associated API-key env-var is not set.

3.9 Prompt Design

Prompts consist of a system and a user prompt. The system prompt, detailed in Appendix B, instructs the model that it is an ICS network-security analyst and outlines how it is supposed to formulate claims:

Only make claims based on the facts given in the bound.

Never make up new attack types.

Never speculate on malice.

Never speculate on causality.

Never speculate on timeframes.

Never speculate on affected assets.

Never speculate on mitigation measures.

Don't claim things are malicious unless this is stated.

Don't claim things are normal unless this is stated.

Attack types in the bound are proxies for actual attacks, and you should not claim attacks happened, but it is OK to repeat the label from the bound.

Rephrase claims freely, but keep asset names, ports, protocol fields etc. always consistent with the bound.

If there isn't sufficient information to determine if something is an attack or normal operation, explicitly say so.

Write three to five sentences.

Write in the language of a professional ICS security analyst.

The user prompt in Appendix B contains the formalized version of the bound as a JSON representation, as well as a human-understandable explanation of the bound, a list of allowed, required, and forbidden claims. The prompt is deliberately very verbose and repeats all rules to avoid attributing undesired model behaviour to deficiencies in prompt engineering. If the model invents an attack type, claims the reverse of what is stated, or does not make a claim that

is supposed to be made, it should not be possible to argue that the prompt was not specific enough and the behaviour could have been avoided with a different prompt. Full prompts are given in Appendix B.

This design choice means that the bar is very high for the language models, where models that pass this test likely perform well even with simpler, less restrictive prompts, and any violations that models commit and that are reported in Chapter 4 represent a lower-bound for violations with simpler prompts. Furthermore, it prevents potential critics from arguing that a verification layer wouldn't be needed with better prompt engineering. The same prompt is used for all tested models and event types, meaning all differences in results stem from the model and the given bound, not from the prompt.

3.10 Sampling and Evaluation Protocol

Prompt selection uses stratified sampling: the prompt builder draws a fixed number of prompts per proxy-event type, currently 25 per type with a fixed seed for reproducibility, so that coverage is comparable across event classes. With four currently-observed proxy-event types this yields 100 prompts per run.

Evaluation is carried out on three samples. The first is a regression sample generated by the synthetic-violation builder: for each of the violation classes V1–V4, it emits one response that exhibits that class in canonical form, as well as a violation-free response, for every sampled prompt-boundary, totalling 500 responses. The second and third samples consist of responses produced by three local models and two cloud models respectively: each model is asked to produce a single explanation for each of the prompts in the sampled set, resulting in 300 local-model responses and 200 cloud-model responses. In total, the final report therefore contains verification results for 1,000 responses, comprising 500 synthetic, 300 local, and 200 cloud, all treated in a uniform fashion.

It is important to use the regression sample to ensure that all possible violations can be recognised by the detector. Some violations are rare, such as V2 direction-inversion violations, so it is unlikely that any given batch of responses from language models will contain an example of every violation class, making it unclear whether absence of a flag is a result of models' good behaviour or inadequate detector coverage. Generating a batch of responses with known ground truth for each violation class, and unit-testing the detector against it, alleviates this concern, and means that absence of a given violation can be taken at face value. We do not

use the regression sample to motivate empirical claims about language models' behaviour, and make sure to keep it clearly separated from actual model-produced samples throughout this thesis.

3.10.1 Evaluation Metrics

To evaluate the performance of the system, a number of metrics have been computed. First of all, it is important to analyse the results for each violation category, and only after that combine them to get an overall result. To compute these metrics the following terms need to be introduced: true positives, false positives, false negatives. A true positive is obtained when a response is classified as belonging to a given class both for the human annotator (main reference) and for the detector, that is, ReasonGuard. A false positive is assigned when the detector considers that an utterance is related to a specific violation while the annotator does not. Lastly, a false negative is attributed to the case in which a response belongs to a specific class for the annotator but the system fails to assign it.

Using these terms it is possible to compute Precision, Recall, F1 score, Accuracy, Cohen's κ . Precision indicates the percentage of true positives among all the responses the detector predicts as positive, that is, belonging to a specific violation. Recall is similar but looks at all the instances in which a response is positive and computes the number of true positives. F1 is the harmonic mean of Precision and Recall and is high only if both metrics are high. These metrics are defined as $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$, $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$, and $\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$; $\text{Accuracy} = (\text{TP} + \text{TN}) / N$. Cohen's $\kappa = (\text{p}_o - \text{p}_e) / (1 - \text{p}_e)$, where p_o is the observed agreement and p_e the agreement expected by chance. The chi-square statistic is $\chi^2 = \sum (\text{O} - \text{E})^2 / \text{E}$ summed over the contingency-table cells, and the associated effect size is Cramér's $V = \sqrt{(\chi^2 / (N \cdot (k - 1)))}$, with k the smaller of the number of rows and columns. To compute the final F1 scores, an unweighted mean of the class-wise F1 scores, that is, a macro-average, was computed. This method assigns the same amount of relevance to all classes and prevents common violations from overshadowing the impact of rare classes. Lastly, accuracy is computed by collapsing all labels into one and computing the fraction of predictions that were correct. Accuracy gives an overall measurement of performance of a classifier, but can be distorted by majority classes especially when dealing with imbalanced data. Therefore, although reported, this metric needs to be treated with more care compared to the others.

Cohen's κ is a statistical measure of agreement between two evaluators, in this case ReasonGuard and the reference of the human annotator, when assigning categorical values to data and corrects this measure for agreement occurring by chance. Values have a common interpretation, where 0 indicates agreement by chance, values below ~ 0.20 indicate slight agreement and 1 indicates perfect agreement. This metric is widely used to assess the reliability of a classifier and imposes higher standards for good agreement, representing the target of the exposé. Therefore, this is the main metric to evaluate the reliability of the system.

The goal of the chi-square test of independence is to analyse whether two categorical variables are related, that is, the violation and model/event type or whether this association is likely to occur by chance. To do so, the p-value computed from the contingency table needs to be less than 0.05, rejecting the null hypothesis of independence. If so, an association between the analysed variables is found.

For the accuracy, Cohen's κ , macro-F1 and both chi-square tests, each response was reduced to a single categorical label by one deterministic rule: a response with no violation is NONE; a response with exactly one violation takes that violation's code (V1–V5); a response with more than one violation is MIXED. The same collapse is applied to ReasonGuard's output and to the human annotation. In the two chi-square contingency tables the classes V2 and V3 were dropped because they had zero count across those response sets.

The previous metrics are used throughout the Chapter 4 to understand how ReasonGuard performs. Moreover, Section 5.3 analyses the substantial gap between target and final scores.

3.11 Manual Annotation

The dataset of 200 manually annotated pairs of explanations is one of the deliverables (M4). It was created with the help of a helper script running in the terminal, displaying one pair at a time and asking for annotation. Annotations were saved after each entry to allow for gradual completion of the whole dataset. Annotations followed these rules:

Does the explanation make any forbidden claims? If so, annotate as such. Otherwise, does the explanation omit any of the mentions required by the bound? If so, annotate as such. Otherwise, has the explanation lost any crucially important specific information? If so, annotate as such.

Otherwise, is the explanation self-contradicting? If so, annotate as such. Otherwise, annotate as none.

The dataset was created by sampling with a guarantee that each violation class is adequately represented, rather than uniformly at random, since some violations are rare. It must be highlighted, however, that, as a limitation of this study, conclusions cannot be drawn for V2, V3 and V5. These classes have few or even no examples in the annotated set. The dataset was sampled from the outputs of all three local models and the synthetic regression set. Of the 200 pairs, 114 were drawn from the synthetic regression set and 86 from the three local models (llama3.1:8b 25, mistral:7b 28, phi3:mini 33); no cloud-model responses were annotated. The annotator did not have access to ReasonGuard's annotations, in order to avoid biasing the annotator towards agreeing with ReasonGuard. The target agreement of ReasonGuard's annotations with the human annotations was Cohen's $\kappa \geq 0.70$ with a macro-averaged F1 ≥ 0.80 .

During the process of annotation, it was discovered that annotating many pairs in a row led to fatigue and subsequently worse quality of the annotation, injecting noise into the resulting dataset. The first attempt to annotate all 200 pairs at once resulted in steadily degrading annotation quality, with agreement going towards 0. As such, this dataset was discarded and the annotation procedure was improved by enforcing a break after annotating each batch of 25 pairs of explanations.

The results of ReasonGuard evaluated on the annotated dataset of 200 samples are not very promising, with accuracy of 0.30, macro-averaged F1 score of 0.19 and Cohen's κ of 0.16, which is significantly worse than the targeted value of $\kappa \geq 0.70$. Moreover, we can see that while the violation class V4 achieves reasonable recall of 0.92, its precision is low at 0.48, indicating that this violation is being over-triggered. On the other hand, violation V1 achieves high precision at the cost of very poor recall, with precision of 0.90 and recall of 0.09. More detailed analysis, including a complete table with scores for each violation and a confusion matrix, will be provided in Chapter 4, together with a more thorough discussion of the results and their relation to the rest of the thesis. Nevertheless, this result, while negative, is important and has conceptual implications which will be discussed in Chapter 5, especially with respect to the use of the proposed solution as a rule-based gatekeeper.

Apart from the dataset of 200 pairs of explanation created for evaluation of ReasonGuard, a different, disjoint dataset of fifty samples was created and sent to a domain expert at the Brno

University of Technology to assess the level of agreement between ReasonGuard and another person and eliminate the potential bias of a single annotator. The expert was not provided with the labels of ReasonGuard, to avoid biasing the annotation. The goal of this annotation is to reach Cohen's $\kappa > 0.70$ on the agreement between ReasonGuard's annotations and those of the domain expert. This annotation was not completed within the scope of this thesis; it remains a planned validation step and is left as future work, and no expert-agreement results are reported here.

3.12 Tracking, Reproducibility, and Data Provenance

All runs are tracked using MLflow in one of five experiments, one each for the bound build, prompt build, synthetic batch, model responses, and violation checker. We track parameters such as the model, temperature, top-p, and prompt-limit, and log the V1–V5 totals as run metrics. Each module in the source has an associated unit-test file, and the test suite can be executed in just a few seconds on the reference workstation.

Appendix C reports the MLflow experiment names, version information, and the commands needed to execute the pipeline, which can be reproduced by anyone by cloning the public repository. We revisit this topic in Section 5.5 and explain why tracking is an important part of regulatory compliance. For example, Article 17 of the EU AI Act imposes record-keeping and monitoring obligations on the providers of high-risk AI systems.

3.13 Research Ethics

This research has been carried out in accordance with the guidelines, rules, policies, and practices of the Institution and its Departments. No human participants were included in the research apart from the author and the supervisors. No personal data was gathered during the study. Data came from public dataset releases by their originators and publishers (for example, on IEEE DataPort) and was used according to their terms of use. No rows of data were invented or altered. The nature of the class under consideration being a proxy is consistently highlighted in the thesis and other deliverables. Negative results such as a low Cohen's κ have been reported without concealment, modification, or aggregation. All endpoints were obfuscated using simple integers instead of IP addresses so as not to reveal functioning networks.

The use of generative AI has been restricted to what is permitted under the SRH's policy on Generative AI and is summarised in Appendix D. This includes its use in code creation and refinement, experimental pipeline creation, data processing, the creation of data annotation

tools, the drafting of references to verifiable facts, and editing. Generative AI has not been used to create any interpretive content in the thesis. All uses have been disclosed and the author retains ultimate responsibility for the accuracy and validity of any AI-assisted outputs.

Because this thesis is a contribution towards improving reliability in the context of safety-critical systems, particular care has been taken to limit the contribution to discussion and demonstration in a responsible way. In particular, the thesis makes clear that ReasonGuard is a research prototype for single-evaluation explanations and has not been deployed as a fully-integrated, certified, real-world solution. No human-in-the-loop studies have been carried out. Discussion of how to manipulate ICS explanation unfaithfulness is kept at the level of promoting its recognition instead of exploitation. All recommendations and contributions are centred around preventing unfaithful explanations from reaching operators by learning when they might be unreliable and acting accordingly — for example, withholding or flagging them as unreliable, auditing their severity, etc. The limitations of the proposed solution, including scenarios where it fails, are made clear in order to avoid misplaced trust. No actionable vulnerabilities of operational, real-world systems are revealed.

4 Results

In this chapter, we empirically evaluate ReasonGuard using IEC-104 data. We follow the evaluation process outlined in Chapter 3. We refer to Appendix C for details on evaluation artefacts. Note that our evaluation is based on the bound schema version 2.1 as a proxy for the official automaton schema from NES@FIT. Hence, the evaluation results are relative to the bound schema and the corresponding classifier introduced in Section 3.3. However, it is possible to repeat our analysis using the official schema and determine how much it differs from the proxy.

Section 4.1 describes the types of violations observed for each model. Section 4.2 compares the violations to the ground truth to investigate how reliable they are. Section 4.3 analyses violation trends for various types of events. Section 4.4 describes the results of our association tests. Lastly, Section 4.5 describes our latency measurements.

4.1 Violation Patterns per Model

Table 4.1 shows the results of our analysis. All data is split by model. For each of the five real models, there are 100 explanations (25 per observed event type), while the synthetic data consists of 500 rule-based explanations. The clean column shows the number of explanations that have no V1–V5 violations. The other columns show the number of explanations that exhibit each violation class. Note that one explanation can violate several violation classes, so that the sum of the columns may be different from the total number of explanations (100/500) minus clean. Notice the difference between Table 4.1, where a response can be included in more than one column, and Table 4.2 and Figure 4.3, where a response is given one label, with responses with more than one type of violation included under MIXED. Therefore, these latter data cannot be directly compared to those shown in Table 4.1.

Table 4.1 Per-model violation summary. Total = explanations produced; Clean = explanations with no violation.

Model	Total	Clean	V1	V2	V3	V4	V5
llama3.1:8b-instruct-q4_0	100	63	22	0	0	20	0
mistral:7b-instruct-v0.3-q4_0	100	33	58	0	0	0	15
phi3:mini	100	2	95	0	0	23	22
gpt-4o-mini	100	28	72	0	0	0	0
gemini-2.5-flash-lite	100	0	0	0	0	100	0
synthetic_rule_based	500	100	100	75	100	400	0

All models exhibit high rates of errors. Looking at Figure 4.1 we can see that in particular phi3:mini fabricates (V1) for almost all prompts and has the largest number of incoherence (V5) violations. For the other models, llama3.1:8b, mistral:7b and gpt-4o-mini, fabrication is the largest error category. Strikingly, gemini-2.5-flash-lite never fabricates but is under-specifying (V4) for all responses.

For an aggregated view we can look at the clean-response rate displayed in Figure 4.2. Here, we see llama3.1:8b performs the best with a rate of 63%, followed by mistral:7b with 33%, gpt-4o-mini with 28%. The rate then drops off to 2% for phi3:mini and 0% for gemini-2.5-flash-lite.

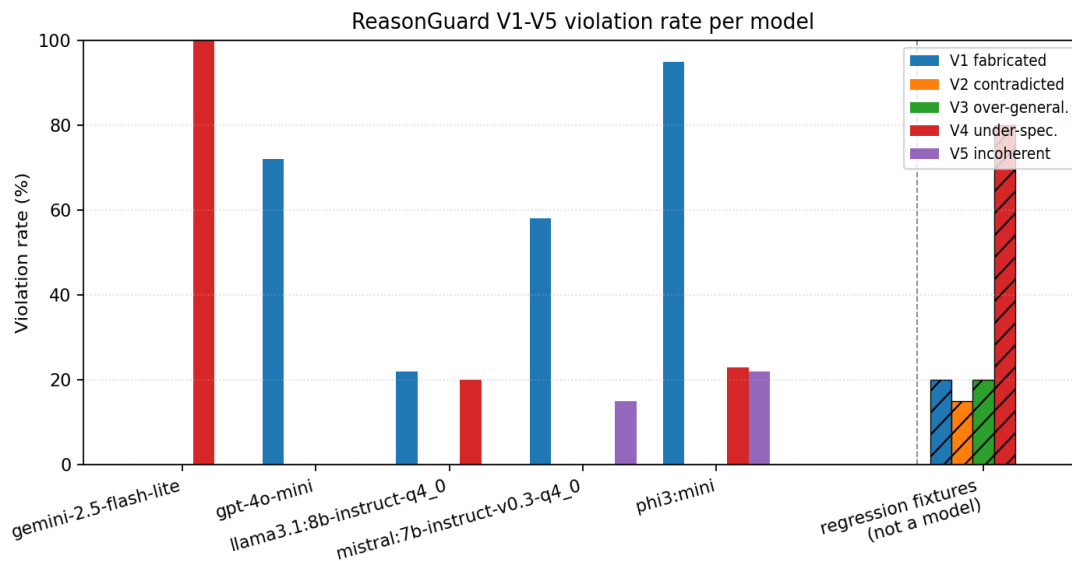


Figure 4.1. *V1–V5 violation rates per model, expressed as a percentage of each model's total responses.*

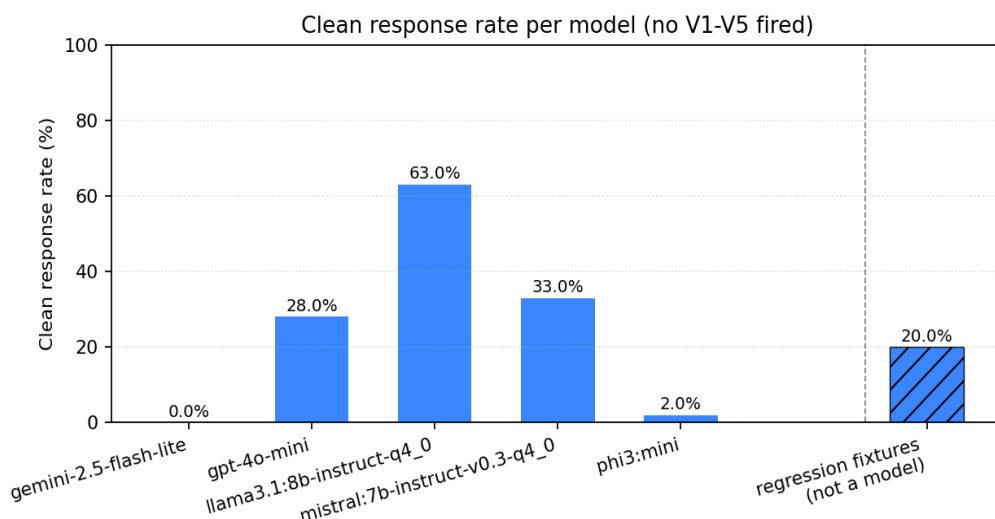


Figure 4.2. *Clean-response rate per model (the proportion of explanations for which no V1–V5 violation fired).*

llama3.1:8b has the highest number of clean results of the local set (63%) and phi3:mini the least. phi3:mini has the most incoherent (V5) outputs, though mistral:7b also shows a non-negligible number.

There are two main takeaways from Table 4.1. Firstly, it is important to consider the last row of Table 4.1, which is synthetic. Here, we see a completely different behaviour in terms of errors. Categories V1–V4 are well represented, which is not surprising since the synthetic engine is set to return one response for each of these classes. This confirms that the detector is sensitive to these error types when they are unambiguously present. This is important since we see that on real models the errors are not well-distributed but concentrated in one or two categories. This suggests that the types of errors are more dependent on the structure of the

model and not just on the difficulty of the task. Particularly noteworthy is the complete lack of both V2 (contradiction) and V3 (over-generalisation). It appears that under a very specific and constraining prompt it is more likely for a model to make statements not grounded on the document (V1) than directly contradicting it, for instance by getting the communication direction wrong. Similarly, it appears more likely to forget to include some aspects of the query (V4) than over-generalise. We shall come back to this point in Chapter 5 since this means that, practically, this is where most of the work is expected to be done.

The second important point to discuss regards the different behaviours of the various models. From these results, llama3.1:8b performed the best and phi3:mini performed the worst. It seems that the larger models are better equipped to perform this task. However, we cannot say with certainty that the models' scale is the driving factor for these results (see Section 6.2). We were able to test three models, but these models not only differed in scale, but also in architecture and training data. Among the cloud models the picture is less clear. It is remarkable that gpt-4o-mini, despite its acknowledged prowess, fabricates heavily. gemini-2.5-flash-lite never hallucinates but also never returns a correct and complete answer. All in all, the results of the cloud models suggest that factors other than model size are important, such as alignment and decoding strategies.

4.2 Detector Performance on the Annotated Set

We test ReasonGuard on the set of 200 human-annotated pairs introduced in Section 3.11. Using all of the pairs in the combined dataset ($n=200$) results in an accuracy of 0.30, an F1 of 0.19 and a κ of 0.16. These results should be kept in mind, but the results from the real-only dataset ($n=86$) should be considered as the primary results, to be compared against the Exposé's targets of $F1 \geq 0.80$ and $\kappa \geq 0.70$: on this real-only dataset, accuracy is 0.19 and κ is -0.01. These real-only results should be treated as the primary benchmark. Table 4.2 provides a detailed breakdown of our results. Overall scores mask large variations in the results on individual classes.

Table 4.2 Per-class detector performance on the 200-pair annotated set.

Class	Precision	Recall	F1	Support
NONE	0.26	0.43	0.33	49
V1	0.90	0.09	0.17	98
V2	0.00	0.00	0.00	17
V3	0.00	0.00	0.00	2
V4	0.48	0.92	0.63	25
V5	—	—	—	0
Macro / Accuracy	0.27	0.24	0.19	acc 0.30

Note. The six per-class rows account for 191 of the 200 annotated pairs; the remaining nine pairs carry a multi-violation (MIXED) human label, shown in the confusion matrix (Figure 4.3) rather than scored as a separate class. Macro-averaged precision, recall and F1 are computed over the six primary classes {NONE, V1–V5} and exclude MIXED. Moreover, per-class results are only meaningful in the case of the classes V1, V4 and NONE. Indeed, on the evaluated dataset of 200 pairs, the detector never labels a response as V2, V3 or V5, and human labels are rather limited in V3 and V5 classes. Note that results presented in this section are computed using synthetic examples designed for testing the detector against known ground truth (114 pairs), as well as real model-generated pairs (86 pairs).

ReasonGuard scores $\kappa = -0.01$ (accuracy 0.19) on the 86 real pairs and $\kappa = 0.27$ on the remaining 114 synthetic fixtures, resulting in a score of $\kappa = 0.16$ for the whole evaluation set (200 pairs). While we report the latter, pooled score for completeness, we recommend focusing on performance on the real responses. In all cases, results on synthetic fixtures should be interpreted with respect to performance on the real responses.

The most common error for V1 can be observed in Figure 4.3, where human-annotated V1 are often classified as NONE or MIXED by ReasonGuard. As a result, although V1 has a precision of 0.90, the detector misses around nine out of every ten fabrications labelled by the annotator and obtains a very low recall of 0.09.

V4 is the other way around, that is, the detector triggers much more often than annotations, resulting in a lower precision of 0.48. However, almost all under-specification labelled by the annotator can be identified by the detector, giving rise to a recall of 0.92.

Other types are not well represented in the annotated sample to obtain reliable estimation. We report their metrics to be zero accordingly.

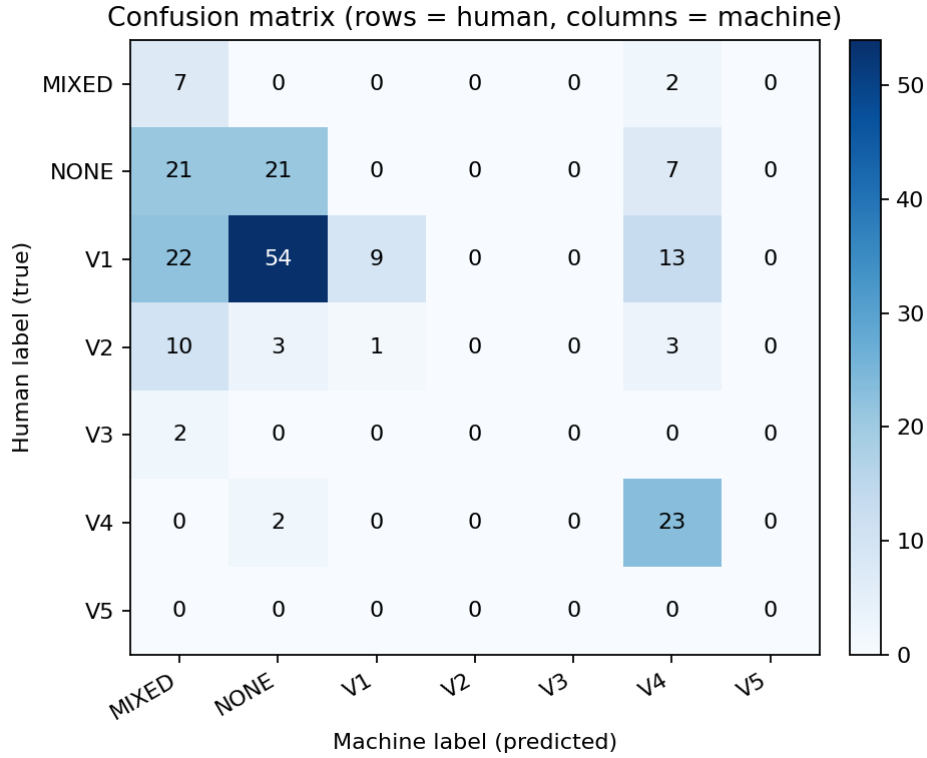


Figure 4.3. Confusion matrix on the annotated set (rows = human label, columns = ReasonGuard verdict).

Overall accuracy can be a misleading indicator of the quality of a classifier, particularly on imbalanced multi-label datasets like ours. It can be helpful to look at precision and recall instead.

For example, although the headline accuracy of ReasonGuard is 0.30, this conceals the fact that ReasonGuard under-detects fabrication (V1) while performing much better on other violation types. Of the responses that the annotator labels as containing a violation of V1, 98 in total, ReasonGuard only labels 9 as being violations of V1, labelling the rest as violations of MIXED (22), V4 (13) or NONE (54). On the other hand, of the 25 responses labelled as violating V4 by the annotator, ReasonGuard labels 23 correctly and the remaining 2 as violations of NONE.

Accuracy can be low on a problem like ours for two reasons: ReasonGuard does indeed struggle to make predictions, but also our definition of the metric penalises ReasonGuard if a response contains multiple violations, in which case the ground-truth label will be the collapsed label MIXED: partial credit is not given, so a situation in which ReasonGuard predicts one of the constituent violations in the ground-truth label counts as a mismatch. Thus, although overall accuracy is an important metric, it is probably the least informative for this particular task.

Overall, these results suggest ReasonGuard to be quite sensitive to V4 (under-specification), albeit with limited precision, but to some extent blind to instances of fabrication.

4.3 Violation Patterns per Event Type

Table 4.3 shows the clean rate per event type for all models. From this, it is clear that the effect of event type on the clean rate is not uniform, and that the direction of this effect depends on the model. Section 4.4 statistically develops these points, and Chapter 5 interprets them.

Table 4.3 Per-event-type clean rate for each model (clean / 25).

Model	Event type	Clean	Rate
llama3.1:8b	function_code_violation	11	44%
llama3.1:8b	network_scan	21	84%
llama3.1:8b	normal_modbus_polling	14	56%
llama3.1:8b	topology_change	17	68%
mistral:7b	function_code_violation	5	20%
mistral:7b	network_scan	6	24%
mistral:7b	normal_modbus_polling	8	32%
mistral:7b	topology_change	14	56%
phi3:mini	function_code_violation	1	4%
phi3:mini	network_scan	0	0%
phi3:mini	normal_modbus_polling	0	0%
phi3:mini	topology_change	1	4%
gpt-4o-mini	function_code_violation	2	8%
gpt-4o-mini	network_scan	12	48%
gpt-4o-mini	normal_modbus_polling	11	44%
gpt-4o-mini	topology_change	3	12%
gemini-2.5-flash-lite	(all four types)	0	0%

From this, we can observe that some event types seem to be globally more difficult: `function_code_violation` achieves a maximum clean rate of only 44%, `phi3:mini` is close to 0% on all of them, Mistral performs worse on the two rarest ones, and `gpt-4o-mini` performs worse

on topology_change (12% clean rate) and function_code_violation (8%). However, there is no globally agreed-upon notion of difficult event types: normal polling achieves a mere 44% for gpt-4o-mini, while Llama achieves 84% on network_scan and Mistral achieves its best result (56%) on topology_change.

We can therefore assert that event type and violation category seem to be statistically associated, although the association depends on the model at hand.

Consequently, although the rare-event hypothesis can explain the relationship to some degree because a lack of contextual cues may lead to more hallucinations, the association between violation category and event type seems to be more nuanced. In particular, the hypothesis that rare events lead to hallucinations cannot be verified in its generality, as, for instance, Llama performs best on network_scan, the second-rarest of the observed event types.

However, keep in mind that we have only considered four event types so far and the results might change once we integrate the Modbus-sourced types. Moreover, with only 25 responses for each cell, the results should be considered indicative until verified on the final dataset.

4.4 Statistical Association

We report the results of two chi-square tests of independence between the violation type (collapsed to a single label per response) and (a) model and (b) the type of proxy event. In both cases we reject the null hypothesis of independence at the 0.1% significance level, suggesting that violations are significantly associated with the model and proxy event type. The test of association with the model has a particularly large test statistic due to categorical differences in failure modes where gemini only fails with the V4 violation, while gpt-4o-mini fails with V1.

Table 4.4 Chi-square tests of independence (violation category vs. grouping variable).

Grouping variable	χ^2	df	p
Model	649.05	16	< .001
Proxy event type	39.09	12	< .001

The model-level test is computed over the 500 language-model responses ($N = 500$), whereas the proxy-event-type test additionally includes the 500 synthetic responses, since these also carry event-type labels ($N = 1,000$). To gauge the strength of association we report Cramér’s V : $V = 0.57$ for the model-level test ($N = 500$) and $V = 0.11$ for the proxy-event-type test ($N =$

1,000), indicating a strong association with the model but only a weak one with the event type. In both contingency tables a minority of cells (20%) have expected counts below five — a consequence of the rare V5 category and of gemini-2.5-flash-lite’s near-degenerate distribution — placing the tables at the boundary of the conventional guideline for the χ^2 approximation, though no expected count falls below two. The χ^2 values, and especially the large model statistic, should therefore be read as indicative of association rather than as precise magnitudes. The primary driver is gemini-2.5-flash-lite, which produces output V4 almost always. As discussed in detail in Sections 4.6 and 5.3, this does not represent a lack of faithfulness, but mostly a tendency to not include a disclaimer about uncertainty or using a proxy.

Moreover, to ensure the trend is robust and not just an artefact of Gemini’s disproportionate share of V4 answers, that is, a 100% rate, the analysis was run again without Gemini, with $n = 4$ models, $\chi^2 = 223.4$, $df = 12$, $N = 400$, $V = 0.43$, $p < 0.001$, and again with both Gemini and the V4 answers removed ($\chi^2 = 188.5$, $df = 9$, $N = 383$, $V = 0.40$, $p < 0.001$). While the strength of the relationship decreases somewhat from 0.57 to between 0.40 and 0.43, it is still quite strong.

Table 4.1 also includes the number of failures in each of the violation categories from Section 3.4. Interpreting these requires taking into account which violations have high vs low severity. We recall that violations of classes V1, V2 and V5 are high-severity, and violations of class V4 are low-severity.

For example, gpt-4o-mini has many (72) failures due to hallucinated entities (class V1), which are high-severity, while Gemini-2.5-flash-lite does not make any high-severity errors, but has many (100) low-severity failures due to lack of nuance (class V4). In deployment, one might make decisions based on severity; consider the simple deployment guideline (1) discussed in Section 5.4.

Under this guideline, phi3:mini’s explanations would almost never be shown to the user because it has 95 failures due to hallucinated entities (V1) and 22 failures due to incoherent (self-contradictory) reasoning (V5), while Gemini-2.5-flash-lite’s explanations would all be shown to the user with a caveat. Both Llama and Mistral have a reasonably high rate of clean explanations, but also high-severity hallucination errors in the rest. Thus, even with such a simple example of deployment considerations, it is important to consider which model is being used along with the criteria on which to gate showing explanations to users.

4.5 Latency and Operational Cost

Table 4.5 shows the average times taken to receive responses to each of the prompts. This was completed on Apple’s M-series CPU (no discrete GPU acceleration available). Models were

accessed via Ollama 0.23.3 and utilised 4-bit quantisation. Interestingly, the maximum latencies are quite large compared to the medians: Llama has 4 instances of latency outliers over 100 seconds with a maximum of 1,022 seconds, phi3:mini has 2 such outliers with a maximum of 1,881 seconds, and Mistral has no outliers (indicating its stability). These outliers strongly affect the mean, making median latency a better performance indicator. phi3:mini has the lowest median latency, followed by Llama and Mistral (10.7s, 21.4s, 23.3s).

Overall, the median times, ranging between 10.7 and 23.3s, are reasonable for most interactive applications. However, there are some extremely slow responses that take between 17 and 31 minutes, and thus, we cannot claim that all requests are interactive.

These slow tail latencies imply that any deployed system should include timeouts and fallbacks.

This is an important factor in its favour for deployment, discussed further in Chapter 5, as it avoids dependency on a cloud-based service to execute the verification element of the pipeline.

Table 4.5 Per-prompt latency per local model.

Local model	n	Mean	SD	Median	Min–Max
llama3.1:8b-instruct-q4_0	100	59.8 s	187.4	21.4 s	12.2–1021.9 s
mistral:7b-instruct-v0.3-q4_0	100	24.2 s	5.2	23.3 s	16.6–39.9 s
phi3:mini	100	35.3 s	193.3	10.7 s	4.7–1881.2 s

4.6 Qualitative Examples

To better understand the results and behaviour of the models, we examine an example output from the larger Llama model that passes verification for a `normal_modbus_polling` bound. The model manages to avoid violating the forbidden-claim constraints and includes useful information about the event's source and destination identifiers, port, and ASDU fields. ReasonGuard assigns a severity level of none, indicating that it believes the response is faithful. The larger Llama model has a clean rate of 63% under the strict-prompt settings.

In contrast, gpt-4o-mini and phi3:mini often fabricated information not present in the original event (V1). This included claiming that the event was of a different, unmentioned attack type, such as describing a control frame as an act of reconnaissance or scanning, resulting in a high-severity response from ReasonGuard. All responses from gemini-2.5-flash-lite failed V4 with a severity of low, leading to its 0% clean rate. However, they did not hallucinate information from the event, as gpt-4o-mini and phi3:mini often did. Instead, they did not give an uncertainty

statement or refer to the proxy, as required. As described in Section 5.3, this reflects a mis-calibration in the evaluation of V4 violations rather than poor performance by the model.

Finally, a minority of phi3:mini's responses were incoherent (V5), for example on network_scan bounds, such as describing the same event as benign and malicious in the same sentence. These also result in high-severity interventions, as it is unclear whether the model understands the situation. Generally, models with greater capacity appear more likely to give responses that are internally consistent and unlikely to violate V5.

4.7 Summary of Findings

Four key findings emerge from this experiment. These will be referred to again in the requirements and discussion given in the following chapter:

F1 Model performance varied greatly, with clean rates ranging from 0 to 63%. Differences between models are statistically significant according to a chi-square test.

F2 In addition to the large variation in aggregate scores between models, their constituent error types also differ significantly. This indicates that each model has a unique error signature and a scalar aggregate score obscures important information that can be used to motivate different verification techniques for each model.

F3 Agreement between ReasonGuard's judgements and human evaluation is relatively poor. For the blended dataset of 200 pairs, we obtain a $\kappa = 0.16$ and an $F1 = 0.19$. These results should be used for context; however, we focus on the performance with real-only examples, since this provides a comparison with chance-level performance: we obtain a $\kappa = -0.01$ and an accuracy = 0.19 for the real-only results. However, it performs particularly poorly for some violation types: it has a high precision but a low sensitivity for fabrication errors (V1), but a high sensitivity and a low precision for under-specification errors (V4). These biases must be accounted for when interpreting ReasonGuard outputs, and could be improved with a fine-tuned LLM or a more effective prompting strategy.

F4 Violation types vary significantly not only between models but also event types. However, this effect differs between models, such that some perform better on attacks and others perform better on normal events. This indicates that there may be scope for tailoring the verifier's behaviour according to the event's context.

Chapter 5 uses these insights to motivate a discussion of the practical requirements of verification. It also discusses the validity of the results.

5 Discussion

This chapter offers a discussion on the results from the previous chapter. First, the implications of the results are described in Section 5.1, after which the threats to the study's validity are discussed in Section 5.2. Subsequently, the negative finding of a low agreement between the tool's output and the human reference is discussed in Section 5.3. Based on the results, guidelines for tool deployment are given in Section 5.4 and related to the EU AI Act in Section 5.5. Last, limitations of this work are presented alongside ideas for future work in Section 5.6.

5.1 Empirical Implications

Overall, ReasonGuard's taxonomy-based evaluation helps understand which models to choose for LLM-based ICS event explanation in the context of our use case and gives insight into potential reasons for failures. ReasonGuard enables a more fine-grained comparison between LLMs for our specific task at hand, as shown in Table 4.1, highlighting strengths and weaknesses of each of them.

First of all, the distribution of violation categories across the five models is statistically significant ($\chi^2(16, N = 500) = 649.05, p < .001$, Cramér's $V = 0.57$), with clean-response rates ranging from 0% to 63%. The breakdown of violations enables a more fine-grained model choice depending on the use case than would be possible by comparing aggregate benchmark results: some models, for example gpt-4o-mini, mainly fail through fabrication, while others, for example gemini-2.5-flash-lite, mainly fail through under-specification. The smaller phi3:mini model exhibits both failure types and produces only rarely clean responses. This granularity also shows that comparing only the aggregated, overall quality of two models can be highly misleading. For example, gemini-2.5-flash-lite has a 0% clean rate, yet all of its failures are low-severity under-specification, whereas gpt-4o-mini has a higher clean rate of 28% but fails almost entirely through high-severity fabrication. Depending on the use case, a model whose errors are consistently low-severity and could potentially be tolerated may be preferable to one that scores better overall but fabricates. Thus, ReasonGuard helps deployers select LLMs more reliably than comparing their aggregate performance, that is, their quality, measured through some scalar. This is particularly important in safety-critical use cases, as ReasonGuard helps understand which failure mode is manageable.

In the context of faithfulness violation detection in particular, it may be interesting to compare the effect of the NLP-based negation filter with a more simplistic negation detector using a list of negating terms, such as do not or cannot, as substrings. Indeed, we initially implemented

such a lexicon-based negation detector but found that it resulted in a substantial amount of false positives of the V1 violation, that is, it did not correctly determine whether a negation in the response invalidated a keyword in the ground-truth description. Replacing this lexicon with the dependency-parse based negation detector from spaCy led to a significant decrease in false V1 violations on the responses of local models. This could be due to negations having a broader scope in the free-form model responses, for example, in complex, multiple-clause negations, making the determination of negation scope through syntactic parsing beneficial for lexicon-based faithfulness detection approaches in general.

Beyond model-level analyses, ReasonGuard also allows for insights into models' weaknesses with respect to certain event types. Indeed, models differ significantly in terms of clean-response rates across the four event types observed in our dataset. This could be related to models being less confident about rare proxy events, leading them to fabricate more often in some models. Other models, for example, the larger Llama model, however, produce the most clean responses about the second-rarest event type.

Finally, from an implementation perspective, our setup enables interactive-speed explanations of ICS events with a local LLM on commodity hardware. This has two advantages for operators of ICSs: First, no confidential data needs to be transferred to a third-party API. Depending on the criticality of the infrastructure, the level of confidentiality may be a legal or economic requirement. Second, verification remains available even if an API cannot be reached. Especially in control rooms, one would anticipate the most interest in such tools during major issues with the monitored system which are more likely to coincide with connectivity issues. Being unable to use such tools exactly when they are needed the most would be inconvenient. However, the trade-off is that using less powerful local models comes with an increased number of faithfulness violations, making verification through ReasonGuard or other methods all the more important.

However, tail latencies are very large and last from 17 to 31 minutes. Although the claim of 'interactive speed' is justified if we consider the median latencies, the aforementioned tail latencies significantly challenge the claim that the system is available when it is necessary to have it right away.

5.2 Threats to Validity

ReasonGuard takes a verdict and uses it as a bound on what an explanation may claim. All claims about ReasonGuard's effectiveness are thus relative to the strength of the given verdict. During testing, ReasonGuard was run in proxy regime where the bound includes the `formal_class` field produced by a component of ReasonGuard itself, an internal classifier. Every artefact produced in proxy regime is labelled as a proxy. Conclusions presented in Chapter 4 should be revisited when ReasonGuard is tested against the official verdict schema.

The V1 to V5 detector is a deterministic function of the given explanation and the bound: given these two inputs, there is no source of variance.

A single annotator scored all 200 rows, which introduced annotator bias. To mitigate this threat, we prepared a blind annotation package with a random sample of 50 prompts for annotation by a domain expert from Brno University of Technology; the resulting inter-rater agreement is left as future work and is not reported in this thesis. The responses from the cloud models are not included in the annotation. The agreement is reported without categories according to the event type. More importantly, as the same person wrote the detection rules and the annotation guidelines, the reference cannot be considered independent. As such, the reported agreement only indicates how close the rules are to the researcher's opinion.

ReasonGuard was empirically evaluated on IEC-104 traffic produced in testbeds of Brno University of Technology. It has not been tested on the remaining data collected (Modbus), nor on types of replay-attack which were not present in our data. However, ReasonGuard should be able to process explanations for other datasets and attacks as well. In particular, the framework should be able to cope with other ICS protocols, since the verdicts are protocol-agnostic, being composed of typed fields, a list of claims (allowed, required, forbidden) and Boolean constraints. Once a verdict is given, the verifier treats them uniformly. Changes needed to accommodate, for example, Modbus, S7 or DNP3 traffic should be limited to components which create a bound from raw rows (feature extraction), and components which translate claims to a set of concepts using lexicon or EntityRuler patterns, that is, where knowledge about the vocabulary used in explanations needs to be incorporated. Note, however, that even though it should be possible to run ReasonGuard on other data, it should not be expected that the measured performance will remain similar. Especially the natural-language components of V1 and V4 calibrations would likely need to be measured anew due to different distribution of events and consequent difference in NL surface.

The sample of 100 prompts per model is large enough to detect substantial differences between models. However, finer-grained statistical analysis of the results, such as comparing success rates on individual event types within a given model, is not always conclusive due to limited statistical power.

In chi-square tests, we used a simplified encoding of verdicts, assigning a single label to a response. We also disregarded labels with zero count, namely V2 and V3.

5.3 Interpreting the Low Agreement

While it is disappointing that ReasonGuard did not achieve the desired degree of agreement, we still think that the results provide an opportunity for fruitful discussion about what happened and why. Overall, the low agreement does not mean that bound-based verification performed poorly, but rather that the implementation of the rules for each class performed differently than expected. Some rules performed better than others.

The most severe disagreement arose in the V4 rule, which is designed to trigger on the absence of any of the five references mentioned earlier. It detected faithfulness violations in $\approx 55\%$ of explanations (every response in which the V4 rule fired), whereas the annotator marked them in 12.5% of explanations. In doing so, it achieved a high recall of 0.92, but produced many false positives. When we compared the two, we found that V4 had been marked in the annotation only where the uncertainty/proxy caveats were absent, whereas the rule had been designed to trigger if any of the four traffic-related pieces of information were absent. This highlights the difficulty of agreeing with humans, even when the annotation guidelines and the rules were authored by the same person.

On the other hand, the V1 rule for Fabrication achieved high precision with a score of 0.90, meaning that if ReasonGuard does identify a claim as fabricated, it is highly likely that the human will agree. However, only 9% of the violations identified as Fabricated by the annotator are retained, likely due to the strictness of the negation scope.

Taken together, ReasonGuard is far from ready to be used as a stand-alone verifier of faithfulness in ICS explanations. In its current incarnation as a set of rules, however, the system could be useful as a first step in a more involved verification process, flagging potential faithfulness violations to be reviewed by a human. Given the high recall in V4 and high precision in V1, a human reviewing the output of ReasonGuard would know that for under-specification (V4) it almost never misses a true case but fires too readily, whereas for

fabrication (V1) it rarely fires but is usually right when it does. In particular, violations flagged as Fabricated would be a lower bound, as there are likely to be more.

In order to automate this process and improve agreement with human annotation, a next step would be to learn a classifier to make the determination for individual claims using a set of examples and human annotations as developed here. This could be used to see if ML-based violation detection would increase agreement with humans, with ReasonGuard's rules serving as baselines for comparison. Agreement with the annotator would need to be tested again after classifier training to validate improvements.

Importantly, these next steps would likely involve developing and applying different thresholds for each type of faithfulness violation. The low agreement with human annotation reflects applying the same thresholding philosophy to faithfulness violations that are quite different. Whereas missing Fabrications are dangerous and might warrant tuning for increased recall even at the cost of lower precision, the goal for Under-specification might be to improve agreement with human annotation by narrowing its scope. In sum, the threshold for each class should be tuned differently depending on the severity of each type of faithfulness violation.

5.3.1 Comparison with Related Work

ReasonGuard achieves a real-only performance of $\kappa = -0.01$ and a blended performance of $\kappa = 0.16$. Comparisons to related work should be made with the real-only number, which leads to a much greater gap, though the blended number may be presented as well for context. As explained in Chapter 2, faithfulness is a hard problem to tackle, however, Honovich et al. (2022) report many cases where learned metrics show good agreement with humans on particular examples. Moreover, the fully-automated pipeline of FActScore's faithfulness metric proposed by Min et al., 2023 performs comparably to their human evaluation within a few percentage points. Learned fact-checking has also seen further improvements: work by Tang et al. (2024) introduces a lightweight model called MiniCheck and shows that fact-checking is feasible with small learned models. There are, nevertheless, two caveats. First, those approaches are learned metrics trained on large annotated datasets. ReasonGuard instead inspects single claims against formal bounds. Second, since those approaches are model-based they are neither deterministic nor auditable: they provide a probabilistic faithfulness score but it is not clear how they arrived at it. Zheng et al. (2023) show that LLM judges exhibit position biases and inconsistencies. This is an issue that cannot be neglected when it comes to using

them alone on safety-critical applications. These two properties of ReasonGuard were deemed relevant and were consciously opted for at the expense of recall and coverage.

Overall, this could be considered a negative result, however, it is by no means a dead end. On the contrary, the results point to a clear line of research. Future work may well incorporate a learned component in ReasonGuard, possibly tailored to improve upon current weaknesses, that is, the poor results for the fabrication error type, while retaining an auditable and deterministic core. Human agreement may very well then play a pivotal role as an acceptance criterion. Furthermore, the experiment itself provides an annotated dataset that may come in handy in this endeavour.

5.4 Operational Deployment Guidelines

The following are the guidelines we propose organisations follow when using the system:

1. Never show explanations with verification statuses V1, V2 or V5 directly to operators, treating them as reliable. These should be flagged to an operator for manual review.
2. Make it easy for operators to retrieve the actual bound given by the detector, perhaps via the canonical verification dashboard.
3. Store the (bound, explanation, verdict) for each alert in an MLflow logging system. Ensure this is backed up periodically, with records retained as needed by applicable compliance regimes.
4. Perform another evaluation whenever the system is updated, such as when the model version, prompt, or quantisation changes, and refrain from releasing this update until satisfactory performance is demonstrated.
5. Never release explanations which make forbidden claims. Ensure that the system does not indicate that the situation is definitely an attack, or definitely not one.

These guidelines are listed roughly in order of relevance, based on how directly they address the goal of making sure that the introduction of the LLM-based explanation does not lead to worse performance on the part of the human.

The first leverages the system's high precision on the high-severity classes: when ReasonGuard does flag V1, V2 or V5, it is usually right, so those explanations should be withheld and flagged for review. We are also transparent about the system's poor recall in some configurations, since it is useful to understand its limitations: given that the recall in V1 is 0.09, many fabrications

go undetected, so even explanations that are not flagged should be shown with caveats rather than treated as fully verified.

The second is meant to combat automation bias, as outlined in Section 2.5. Showing the underlying evidence which the explanation is attempting to clarify may help an operator to draw conclusions based on evidence, instead of the LLM-generated text. We would expect the same motivation applies to communicating uncertainty.

The third and fourth guidelines are less specific to ICS, but are useful for accountability and ease of use.

The final guideline reflects our use of the proxy schema, adopting the honest posture the results of Section 5.3 demand, which refrains from making definitive claims.

We emphasise that our assumption throughout this work is that the detector is not good enough for autonomous operation: the goal is to introduce explanations to make it easier for humans to act on its findings. With these guidelines, a human should be no worse off for their introduction.

5.5 Mapping to the European Union AI Act

The monitoring of critical infrastructure is listed as a use case under high-risk AI systems in Annex III, point 2. ReasonGuard itself is a grey area, as it has no influence on actual traffic and no connection to a system that manages physical gating, making it hard to definitively declare it a "safety component". However, if a human analyst takes action based on the violation verdicts given by ReasonGuard, this changes the picture quite a lot. Therefore, in our opinion, ReasonGuard should be built according to these requirements.

One of these requirements, laid out in Article 13, is transparency. Deployers of high-risk systems should be provided with sufficient information to understand them. This includes an intelligible presentation of the system, an outline of the system's performance, including precision and recall metrics, and known limitations. These transparency needs are addressed by deployment guideline (2), which makes the underlying bound available to the operator, and by the per-class performance disclosure in Section 4.2, which gives precision and recall numbers per class. If one were to disregard the per-class metrics, ReasonGuard achieves an aggregated macro-F1 of 0.19 or an overall accuracy of 0.30, which one might be tempted to present as adequate. However, these scores would hide that ReasonGuard currently misses a substantial number of samples in violation class V1, with a recall of 0.09, whereas it performs comparatively well in this category with a precision of 0.90. At the same time, ReasonGuard

over-triggers on violation V4, where it achieves a comparatively low precision of 0.48 and comparatively high recall of 0.92. As discussed in Chapter 4, missing violations in class V1 is much worse for ReasonGuard's use case than over-triggering for other violation types. Disclosing the per-class metrics allows deployers of ReasonGuard to get a more fine-grained impression of the system's strengths and weaknesses.

Article 17 asks for quality management and governance. This includes having a testing and validation protocol in place, governance structures to make sure that the model performs well on the intended use case, logging capabilities to keep records, and monitoring to make sure that performance does not degrade over time. These needs are addressed by deployment guidelines (3) and (4), which cover logging and periodic re-evaluation.

This project followed a lot of the above-mentioned best practices during development, although, admittedly, with the lower demands one would expect for an academic software project:

- Testing and validation were done on the manually annotated dataset. This could be repeated regularly to monitor ReasonGuard's performance. However, during this project this was treated as a one-time effort.
- Record keeping was done throughout the project. An AI-disclosure log was maintained. MLflow was used to document machine-learning pipelines. Even this thesis' underlying code repository can be regarded as a record-keeping resource, as it documents the decisions that were made throughout the project and their rationales.
- Performance issues found during analysis were investigated. It was, for example, discovered that ReasonGuard triggers too often on violation type V4. The problem could be localised to a single rule. The impact of removing it was measured. Although removing the rule would bring ReasonGuard closer to human baselines in terms of macro-F1 score, this did not justify its removal. The decision was made to keep the rule as is and revisit it after submission.

To use ReasonGuard as a full-fledged system, the aforementioned steps need to be elevated from isolated efforts into a monitoring and validation pipeline with clear structures around them. For example, it should be made clear who is responsible for acting on validation results and what kind of decisions should be made. While these are admittedly major changes that have to be made, we think it is a strength of this project to make them apparent and turn

weaknesses of ReasonGuard as an academic project into specifications for ReasonGuard as a production-ready system.

5.6 Limitations and Future Work

The use of a proxy schema for `formal_class` makes all related results dependent on the behaviour of this proxy; replacing it with the official NES@FIT automaton schema is the single highest-priority follow-up task and is an incremental change to the existing artefacts, which can be regenerated deterministically once the official schema is plugged in.

The rule-based V1 to V5 classifier trades performance for interpretability and auditability, as discussed in Section 5.3; a learned version can be trained using the 200-pair annotation as a starting point, re-using the same MLflow-based evaluation infrastructure.

Results were obtained for three local and two cloud LLMs and four of the five standard event types; the remaining one, replay-attack, is absent from the IEC-104 sample and can be obtained from the Modbus dataset using the same infrastructure. More models and events would be needed to make broader claims about LLM capabilities.

Finally, the current results do not take into account how V1 to V5 annotations affect operator decisions relative to the same alerts without annotations; an operator-in-the-loop study is needed to answer this question before the approach could be considered ready for real-world use.

6 Conclusion

6.1 Summary of Contributions

The main technical contribution of this thesis is the ReasonGuard framework, which verifies the faithfulness of AI reasoning at runtime by checking explanations against structured outputs from formally verified detectors. ReasonGuard is motivated by the fact that outputs of formally verified detectors provide a much tighter approximation of ground truth than the unstructured documents that have been used for faithfulness checking thus far. We define five types of reasoning violations formally and verify them using a combination of lexicon-based and syntactic detectors. We make ReasonGuard available as a Python package. We evaluated ReasonGuard with five LLMs on the IEC-104 dataset created by the NES@FIT research group at Brno University of Technology, on four different types of proxy events, and found evidence using chi-square testing that reasoning violation classes are significantly related to both proxy event type and LLM. Based on these results, we offer a checklist of five items to consider before deploying reasoning-enabled LLMs and relate the contributions in this thesis to relevant portions of the EU AI Act.

On the methodological front, this thesis offers an example of a well-documented negative result, one in which we fell short of our target evaluation results. We attribute the shortfall to a single rule in particular, an example of the sort of evidence one might be asked to produce when operating under a formal quality-management process. Furthermore, although aggregate evaluation results were reported throughout the thesis, we emphasise that for safety-critical systems it is more important to understand detector failures on a more granular per-class basis. Aggregate results may be preferable from an academic standpoint as they tend to be more flattering, but these alone would be considered a significant omission in a safety-critical application.

6.2 Answering the Research Questions

The answer to the main research question — can formally verified system outputs be used as runtime bounds for detecting and classifying AI reasoning violations, and does such a framework produce actionable, operationally reliable violation detection in safety-critical AI deployments? — can be interpreted as a negative result: although violation detection via runtime bounds is feasible and yields an interpretable outcome, its poor reliability in this proxy evaluation prevents it from being used in its current form. Because the official NES@FIT automaton schema was unavailable, this evidence is relative to the proxy regime and does not

validate the framework against the official formal schema. In particular, this thesis's contributions comprise a sound and well-founded framework presented in Sections 3.3 and 3.4, a working, thoroughly-evaluated implementation in Chapter 4, and a roadmap for making violation-detection reliable enough for safety-critical use, detailed in Sections 5.4 and 5.5.

With regard to the sub-questions, the thesis's conclusions are:

RQ1 (bound representation): A JSON schema for representing a bound, including metadata, features, allowed/required/forbidden claims, and machine-checkable constraints was introduced in Section 3.3. While this schema can be considered a proxy for the final schema as discussed in Chapter 3, it can be used to verify AI outputs. Further, it provides a solid starting point from which to implement improvements, making an upgrade to the official JSON schema the number-one priority for improving this thesis's implementation.

RQ2 (violation taxonomy): The violation taxonomy with violation types V1 to V5 was introduced in Section 3.4. Observations made during the empirical evaluation in Chapter 4 suggest that V1 and V4 violations are the most prominent, V3 violations occur rarely, V2 violations occur primarily for the synthetically generated bounds, and V5 violations seem to occur more often for the smallest (local) LLM.

RQ3 (detector accuracy): The violation-detection prototype was found to yield a high level of precision and a lower level of recall for V1 violations and a high level of sensitivity and a lower level of precision for V4 violations in the evaluation for the 200-pair evaluation dataset detailed in Chapter 4. However, the detector's macro F1 (0.19) and κ (0.16) scores indicate that improvements are needed before it can be used for violation-detection in a reliable, autonomous manner. However, the reliability of the cloud models with respect to the human ground truth and the performance on individual events with respect to the human ground truth are left to future research. For the combined dataset, the reference value is $\kappa=0.16$, but our main result (real-only) is $\kappa=-0.01$ and accuracy=0.19, which is what answers RQ3 for the local models.

RQ4 (conditions for violation): Chi-square tests revealed that the types of violations made by LLMs significantly depend on the (a) LLM used and the (b) category of event for which it generates an output. Moreover, among the three local models tested (at a fixed 4-bit quantisation), the smaller models tended to produce higher violation rates; because these models also differ in architecture and training data, model size cannot be isolated as the cause, and the cloud models do not provide a clean size ordering.

RQ5 (operational guidelines): Guidelines for safely using violation-detection in the form of a five-item deployment checklist, which includes a mapping of its items to provisions of the EU AI Act, were introduced in Sections 5.4 and 5.5. It can be used by operators when acting upon a bound's verdict with respect to the taxonomy introduced in Section 3.4.

In summary, although violation-detection could be shown to be feasible, the prototype's performance fell short of the targets. However, given its importance as a stepping-stone towards reliable, autonomous violation-detection, its performance has been honestly reported throughout Chapter 4.

6.3 Future Work

First and foremost, ReasonGuard should be extended with support for the official NES@FIT schema described in Section 5.6. The classifier used as a proxy in this thesis introduces some bias in the results and may be performing differently than the original. With support for the official schema, all artefacts presented in this thesis could be deterministically regenerated and the entire evaluation repeated to determine the difference in performance.

Secondly, the planned machine learning component for the per-claim classifier could be added. The dataset of 200 explanation/claim pairs annotated as correct/incorrect provides some starting point for training. This change would retain the existing rule-based component of the per-claim classifier and add a learned component. The rules provide a good starting point that can be verified by humans, but may be lacking in recall. This is evident in the low recall of fabrications in V1. Ideally, the ML component would increase the recall of claims like those identified by V1, but not suffer from the same over-triggering problem exhibited by V4. With the MLflow evaluation already in place to compare human agreement on correct/incorrect labels, it would be relatively simple to use it again to determine whether adding the ML component improves agreement with humans on these labels.

A user study is another interesting extension of this thesis. ReasonGuard provides useful, categorized, locally-available checks on explanations generated on commodity hardware. The tool provides valuable insight into what went wrong with an explanation if an incorrect one is generated, but cannot be relied upon to operate without a human. To really understand the utility of the system, an experiment should be performed in which an operator is put under some degree of time pressure and shown explanations/ReasonGuard verdicts, such as the number of severe problems detected according to checks V1–V5. Then their decision/action taken could be compared to a similar scenario without ReasonGuard verdicts to determine if

ReasonGuard leads to better decisions. This would demonstrate ReasonGuard making a process safer. As described in Section 5.3, this thesis measures the agreement between ReasonGuard and humans on individual explanations, which may correlate with making a process safer. However, there is no direct evidence of a safety improvement.

Lastly, ReasonGuard has been demonstrated to work with one protocol and one kind of behaviour that might be deemed malicious. More data from other protocols could be incorporated into ReasonGuard to determine if the architecture is sufficiently generalisable and could be applied to other problems. For instance, datasets have been collected for the Modbus, DNP3 and S7 industrial control system communication protocols. It is unknown whether ReasonGuard checks would transfer to these new protocols, but they seem worth investigating. These datasets also provide another avenue for improving upon the limited coverage of canonical events presented in Section 5.2. Ideally all five canonical event types would be empirically evaluated, but ReasonGuard as presented in this thesis only achieves this by proxy. Specifically, function-code-violation and replay-attack events should be easy to identify in Modbus data and ReasonGuard verdicts generated for them.

ReasonGuard makes assumptions about the environment it operates in, such as operating within the trust boundary, but has been shown to detect many, though not all, incorrect claims in NES@FIT-generated explanations. One strength of this thesis is acknowledging these weaknesses and shortcomings. Providing information about the shortfalls of ReasonGuard provides clear targets for subsequent work. This is better than burying poor fabrication recall (V1) where it is hidden by averaging with the good recall on other classes, such as V4. Especially in safety-critical applications, overstated claims of reliability are much worse than openly-admitted shortfalls.

References

- Clarke, E. M., Grumberg, O., & Peled, D. A. (2001). *Model checking*. MIT Press.
- de Silva, G., Ryšavý, O., Matoušek, P., & Švéda, M. (2013). On formal reachability analysis in networks with dynamic behavior. *Telecommunication Systems*, 52(2), 919–929.
- European Parliament & Council of the European Union. (2024). *Regulation (EU) 2024/1689 of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- Ferrando, A., Gatti, A., & Mascardi, V. (2023). RV4Rasa: A formalism-agnostic runtime verification framework for verifying chatbots in Rasa. In *Proceedings of the 6th International Workshop on Verification and Monitoring at Runtime Execution (VORTEX 2023)*. ACM. <https://doi.org/10.1145/3605159.3605855>
- Havlena, V., Matoušek, P., Ryšavý, O., & Holík, L. (2023). Accurate automata-based detection of cyber threats in smart grid communication. *IEEE Transactions on Smart Grid*, 14(3), 2352–2366.
- Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kukliansy, D., Cohen, V., Scialom, T., Szpektor, I., Hassidim, A., & Matias, Y. (2022). TRUE: Re-evaluating factual consistency evaluation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)* (pp. 3905–3920).
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2), Article 42. <https://doi.org/10.1145/3703155>
- Leucker, M., & Schallhart, C. (2009). A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5), 293–303.
- Liang, G., Weller, S. R., Zhao, J., Luo, F., & Dong, Z. Y. (2017). The 2015 Ukraine blackout: Implications for false data injection attacks. *IEEE Transactions on Power Systems*, 32(4), 3317–3318.
- Matoušek, P., Ryšavý, O., & Grofčík, R. (2022). *ICS dataset for smart grid anomaly detection* [Data set]. IEEE DataPort. <https://doi.org/10.21227/1trw-n685>

- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 1906–1919).
- Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 12076–12100).
- Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230–253.
- Ryšavý, O., & Matoušek, P. (2021). *Modbus dataset for ICS anomaly detection* [Data set]. IEEE DataPort. <https://doi.org/10.21227/e1bc-3w91>
- Taleb, R., Hallé, S., & Khoury, R. (2023). Uncertainty in runtime verification: A survey. *Computer Science Review*, 50, Article 100594. <https://doi.org/10.1016/j.cosrev.2023.100594>
- Tang, L., Laban, P., & Durrett, G. (2024). MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 8818–8847).
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.

Appendix A Reasoning-Violation Taxonomy in Full Detail

This appendix reproduces the V1–V5 taxonomy with the rule-based detection logic, the lexicon term lists, and the safe-negation patterns used by the production checker. The canonical source of truth is the `reason_guard_checker` module.

A.1 V1 — Fabricated Reasoning

Definition. The AI asserts facts, causes, or entities not present in, or derivable from, the formal bound. Under the proxy bound every `*_claim_allowed` flag defaults to false, so any unsupported attack, mitigation, causal, or numeric assertion fires V1. Detection applies a two-layer check: a spaCy dependency-parse filter walks from the term token up the dependency tree and treats the term as safe if any ancestor has a negation child (not, no, never, neither, nor, without) or if the sentence contains “proxy” or “schema”; a substring-based safe-negation list provides the same protection when spaCy is unavailable. Both layers are conservative — a term is treated as unsafe whenever neither layer can match it to a negation. The numeric check compares each integer in the response against the bound's feature values plus the constants {104, 2404} plus the observed timestamp, flagging values outside that set that exceed 1,000.

A.2 V2 — Contradicted Reasoning

Definition. The AI explicitly contradicts a fact established by the bound. The detector checks two contradiction cases: direction inversion (the response uses the opposite of the bound's `communication_direction` without also stating the bound's direction) and identifier substitution (the response places the `destination_id` value next to the word “source” without containing the `source_id` value). Further contradiction cases are added as they are observed in annotation.

A.3 V3 — Over-Generalised Reasoning

Definition. The AI produces a correct high-level category but loses formally established specifics the operator needs. The detector fires V3 when any normality term (normal traffic, benign, routine, typical, standard communication, no specific concern, normality) appears together with fewer than three of the formally observed specific values (`source_id`, `destination_id`, `srcport`, `dstport`, `asdu_type`, `asdu_cot`).

A.4 V4 — Under-Specified Reasoning

Definition. The AI omits formally established information operationally required for a correct response. The detector fires V4 when any of the required references — source, destination, port, ASDU, and the combined uncertainty/proxy clarification — is missing. Missing

references are reported as individual reason strings so the operator can correct the explanation rather than merely learn that V4 fired. As Section 5.3 discusses, this any-of-five rule is the principal source of the detector's over-triggering against the human criterion.

A.5 V5 — Incoherent Reasoning

Definition. The AI explanation is internally self-contradictory, independent of the formal output. The detector uses spaCy sentence segmentation to find sentences containing both members of a canonical conflicting pair, skipping sentences that contain a negation marker. Conflicting pairs include: normal/malicious, normal/attack, benign/malicious, routine/attack, isolated/entire-network, single-device/entire-network, and client-to-server/server-to-client.

Appendix B Prompt Specifications

Every LLM in the evaluation receives the following production prompt, with a single formal bound substituted into the user section.

B.1 System Prompt

```
You are an industrial control systems network security analyst.
Your task is to explain a formally bounded IEC-104 network
observation in precise operational English.
Strict rules:
1. Use only facts explicitly provided in the formal bound.
2. Do not invent attack types, malicious intent, causality,
   timing patterns, affected assets, or mitigation actions.
3. Do not claim the observation is confirmed normal or confirmed
   malicious unless the formal bound explicitly states that.
4. Treat proxy formal classes as observation labels, not verified
   attack labels.
5. Stay consistent with the source identifier, destination
   identifier, ports, packet size, ASDU fields, protocol hint,
   and communication direction.
6. If evidence is insufficient for a security conclusion, say so.
7. Write 3 to 5 sentences.
8. Use professional analyst language.
```

Listing B.1. Production system prompt, constant across every model call.

B.2 User Prompt Template

The user prompt is generated from a single formal bound and embeds the event metadata, the formal observation, the observed features, and the three claim lists (allowed, required, forbidden) before the instruction to write the explanation. Allowed claims are atomic statements derivable from the observed features; required claims state that the explanation must mention the source identifier, the destination identifier, the ports, the ASDU fields, and the proxy nature of the formal class; forbidden claims state that the explanation must not assert a confirmed attack, confirmed malice, an arbitrary attack class, additional devices, a causal explanation, an unobserved timing pattern, a mitigation requirement, or verified normal traffic.

B.3 Generation Parameters

Every local model is called with temperature 0.1, top-p 0.9, an Ollama 0.23.3 4-bit-quantised backend, and a 180-second HTTP timeout, all persisted in the pipeline configuration so any run can be reproduced from the configuration alone. The cloud runner uses the same parameters where the vendor API supports them, with a 600-token cap.

Appendix C Reproducibility Notes

The full source code, the evaluation scripts, and the manuscript are released under the MIT licence at <https://github.com/bilalovrevan/reasonguard>. The live verification dashboard is deployed at <https://reasonguard-dashboard.streamlit.app>. The annotated evaluation dataset and the published bound batch are released on Zenodo at <https://doi.org/10.5281/zenodo.22724731>.

C.1 Software Versions

- Python 3.10+ (3.14 on the reference workstation); pandas ≥ 2.0 ; numpy ≥ 1.24 .
- spaCy 3.8 with en_core_web_lg 3.8; scipy ≥ 1.11 for the chi-square tests.
- MLflow 2.10+ (SQLite store); Ollama 0.23.3 (4-bit, CPU-only); matplotlib 3.7+; streamlit 1.30+; pytest 7.4+; ruff for linting.

C.2 Reproduction Commands

```
git clone https://github.com/bilalovrevan/reasonguard
cd reasonguard && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m spacy download en_core_web_lg
# local models via Ollama:
ollama pull llama3.1:8b-instruct-q4_0
ollama pull mistral:7b-instruct-v0.3-q4_0
ollama pull phi3:mini
# end-to-end run (25 prompts per event type):
scripts/run_full_pipeline.sh 25
```

Listing C.1. End-to-end reproduction from a clean clone.

C.3 MLflow Experiments

Five MLflow experiments hold the run history: `reasonguard_formal_bounds` (one run per bound build), `reasonguard_prompts` (one run per prompt build), `reasonguard_synthetic_responses` (one run per synthetic batch), `reasonguard_ollama_responses` (one run per model batch, with model, temperature, top-p, and pilot-limit as parameters and `ok_count`, `ok_rate`, and `mean_elapsed_seconds` as metrics), and `reasonguard_violation_checker` (one run per checker invocation, with the V1–V5 totals as metrics).

Appendix D Generative-AI Tool Disclosure

This appendix is included in fulfilment of the SRH “Generative AI in Research” policy, which requires that all AI tools be disclosed with what they were used for. Generative AI was *not* used to author the interpretive prose of the thesis (the passages marked in red); its use was limited to the permitted categories below — code and pipeline engineering, data processing, annotation tooling, factual-reference drafting, project planning, document assembly, and language feedback on author-written text. The author takes full responsibility for verifying the accuracy of all AI-assisted material.

Table D.1 Summary of generative-AI tool use.

Month	Category	Description of Assistance
April	Pipeline / Coding (AI-assisted coding — explicitly permitted for engineering work)	Early-stage support with the pipeline architecture design — structuring the flow (formal_bound_builder → event_type_classifier → prompt_builder → LLM runners → checker), i.e. light architectural/coding input rather than full implementation
May	Pipeline / Coding	Support during the implementation phase — building out the components of the pipeline in code
June	Pipeline / Coding	Refinement and improvement of the existing pipeline — debugging, adjustments, iteration on what was built in April–May
July	Pipeline / Coding	Support in preparing the Streamlit dashboard (input: formal output + LLM explanation → violation report)
July–August	Editing / Brainstorming (structural guidance)	Helped establish an overall structural perspective for writing the thesis, and gave some directional pointers/suggestions for how to approach it